LAPORAN PRAKTIKUM

POSTTEST 4

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhamad Radja Nur Akbar (2509106012)

Kelas (A1'25)

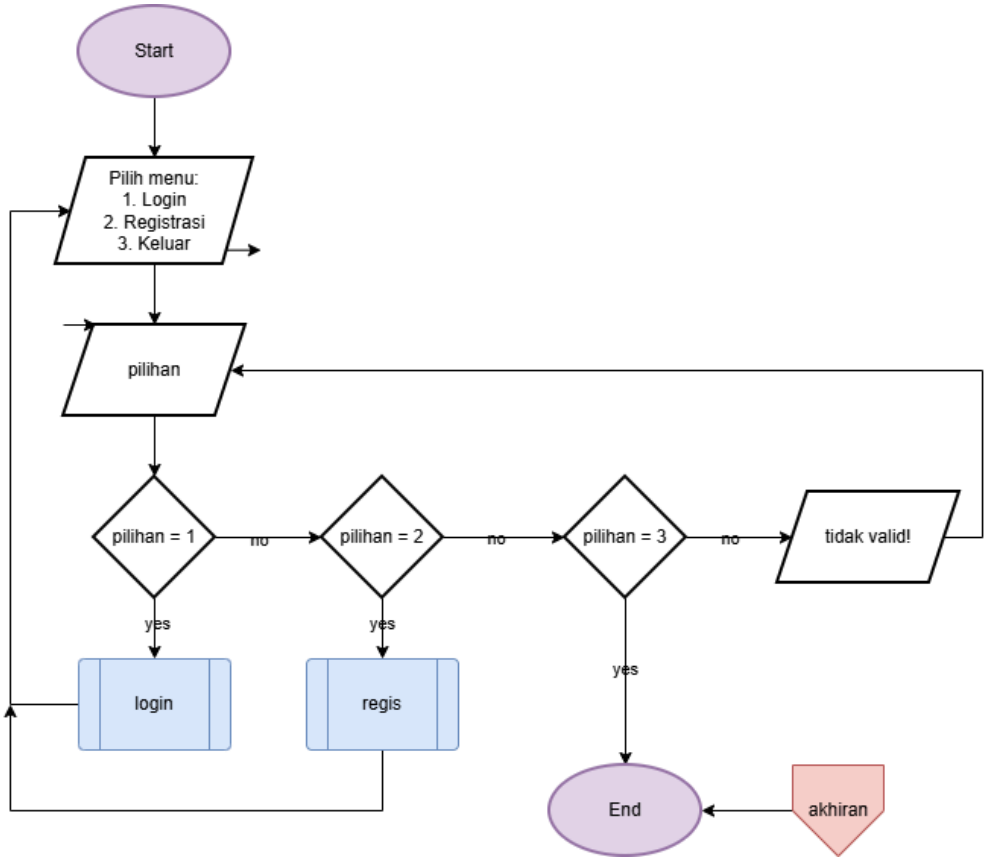
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

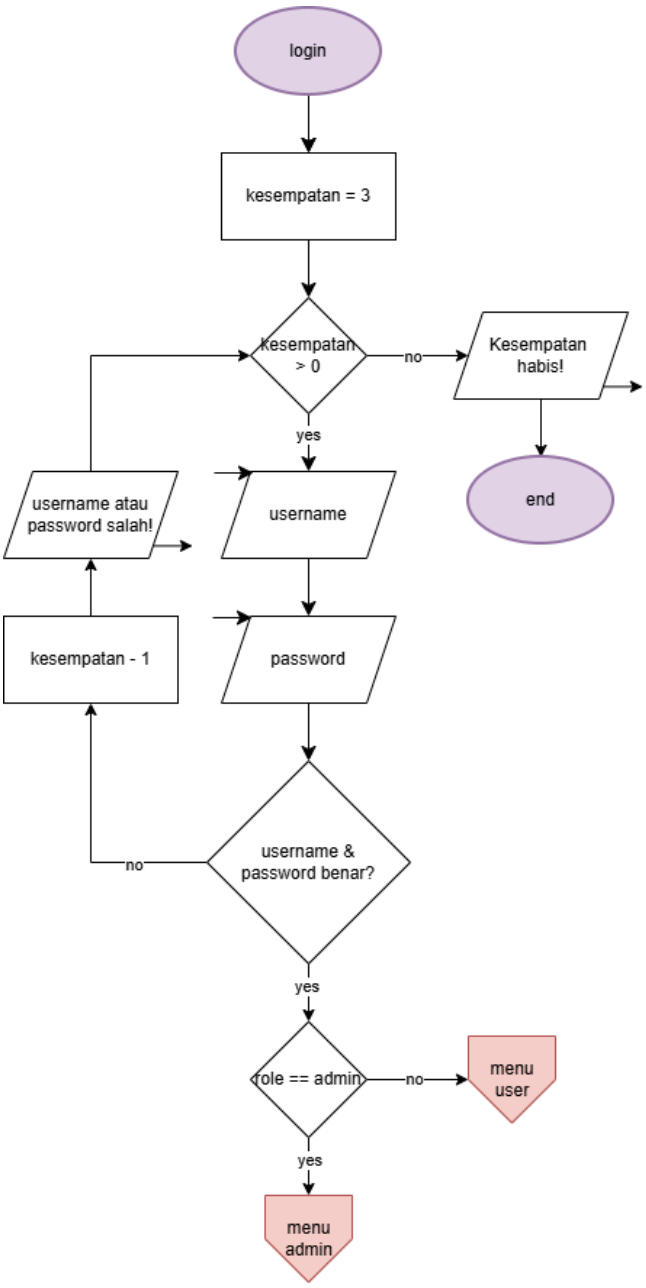
2026

1. Flowchart



Gambar 1.1 Menu Autentikasi

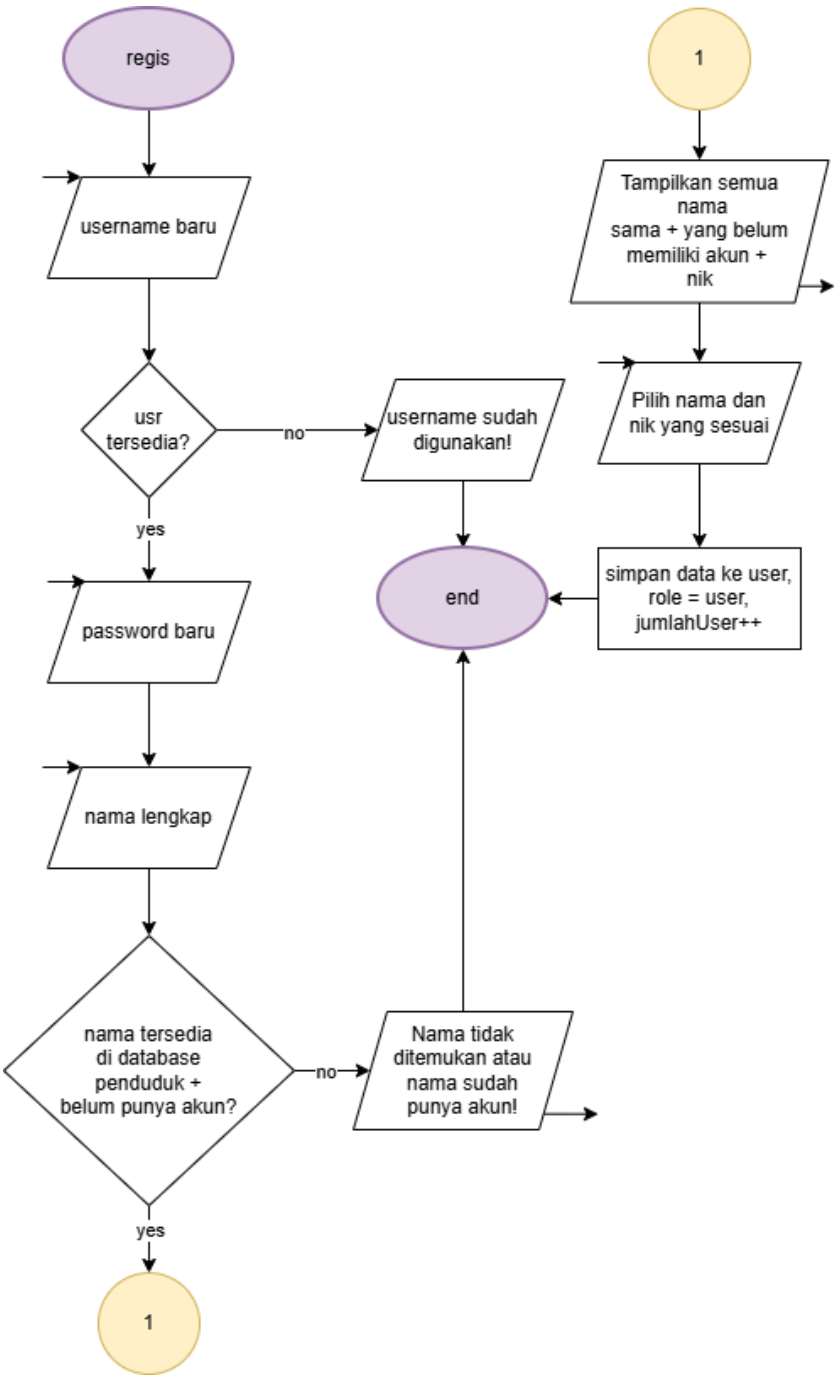
Tampilan awal dari program ini adalah menu autentikasi. Pengguna bisa memilih *login*, registrasi, atau keluar program. Kemudian buka fungsi yang sesuai.



Gambar 1.2 Fungsi Login

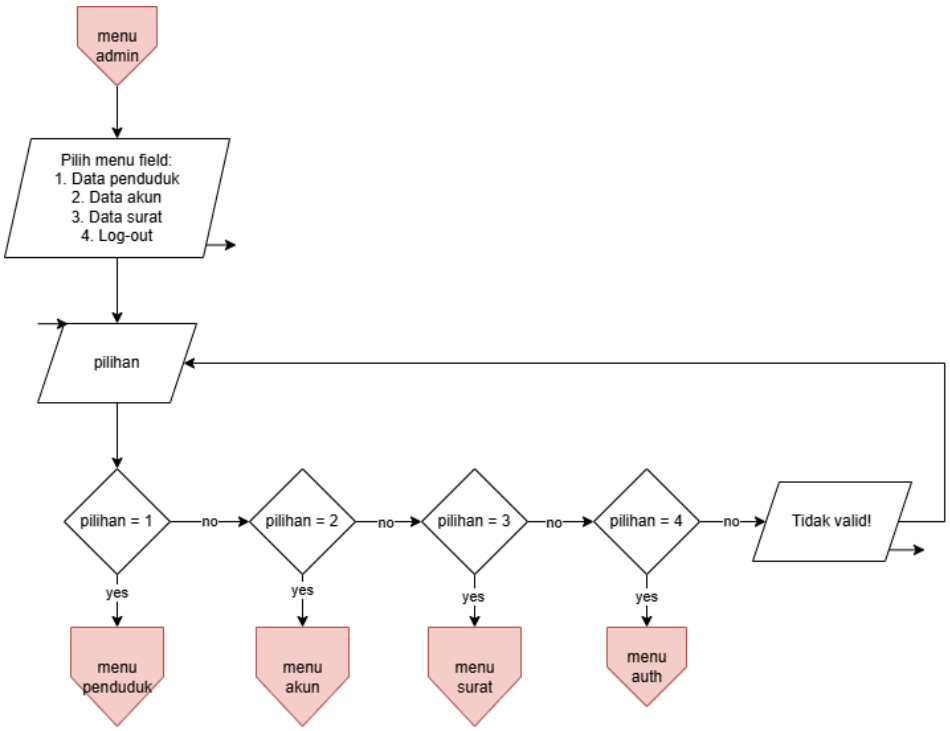
Dalam fungsi *login*, pengguna punya 3 kesempatan untuk *login*, setiap kesalahan akan mengurangi kesempatan sebanyak 1. Jika habis program berhenti.

Ketika *login* sudah tepat, *role* dari akun tersebut dicek untuk membawa pengguna ke menu yang sesuai. Ada dua, menu admin dan menu *user.*



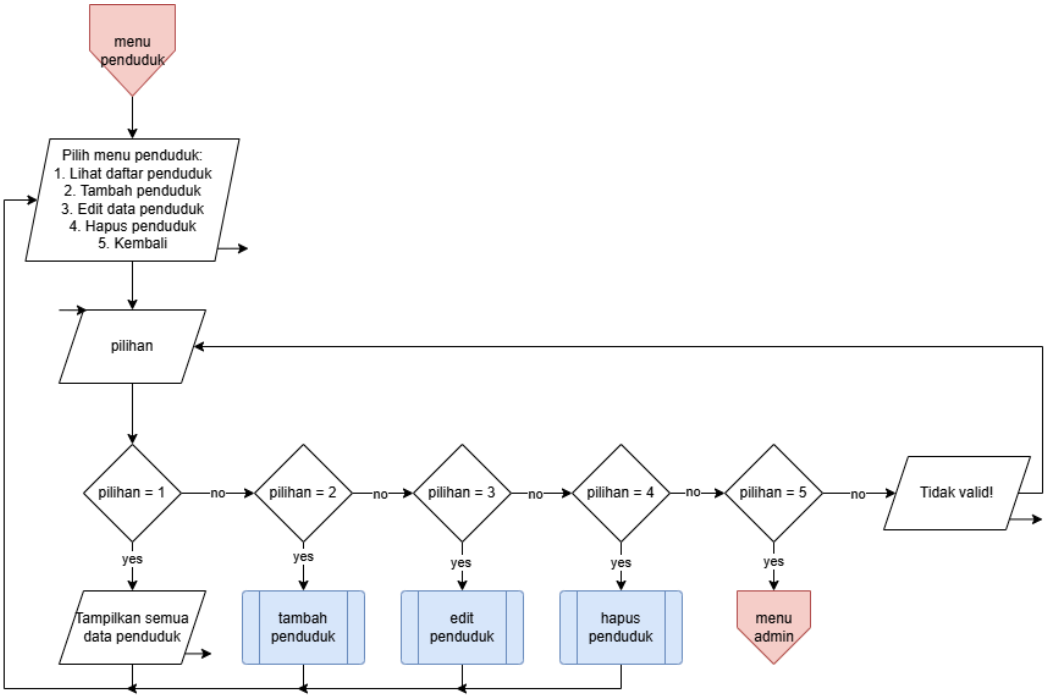
Gambar 1.3 Fungsi Registrasi

Dalam fungsi ini, masukkan *username* baru yang akan ditolak jika *username* sudah digunakan. Masukkan *password* dan nama lengkap. **Catatan:** pengguna hanya bisa registrasi jika mereka terdaftar di *database* penduduk. Jika terdapat nama yang sama dari *database* maka semua nama tersebut ditunjukkan beserta NIK supaya pengguna bisa memilih data yang tepat. Terakhir simpan *username* dan *password* dengan *role* bawaan adalah *user.*



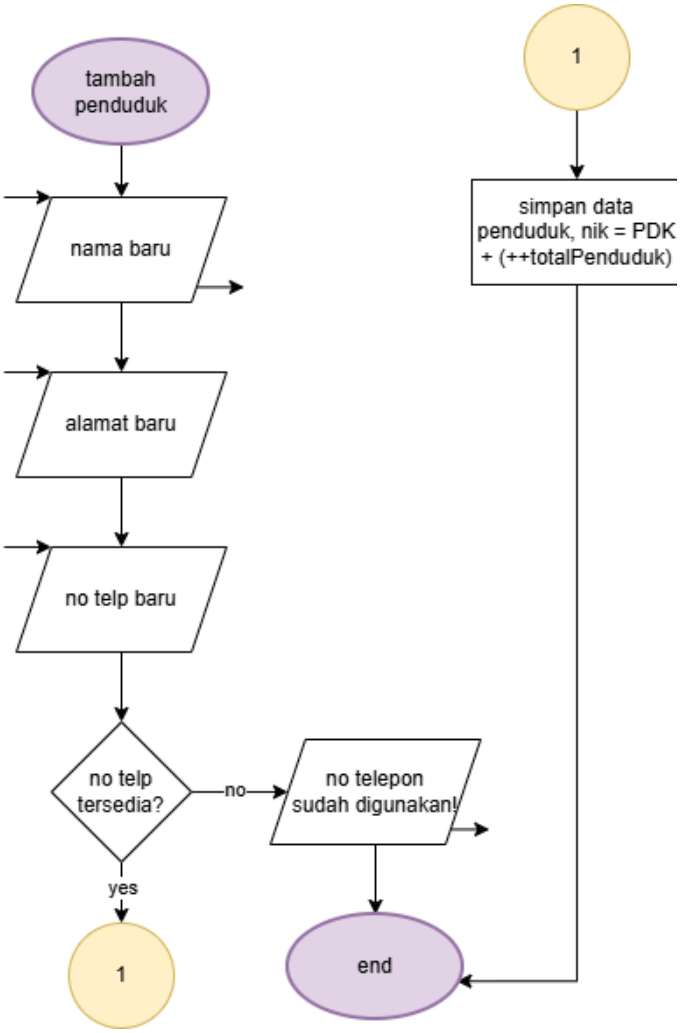
Gambar 1.4 Menu Admin

Admin dapat melakukan 3 jenis CRUD dalam kepengurusan balai kota, yaitu mengurus data penduduk, data akun, dan data surat. Admin hanya perlu memilih salah satu menu untuk melanjutkan.



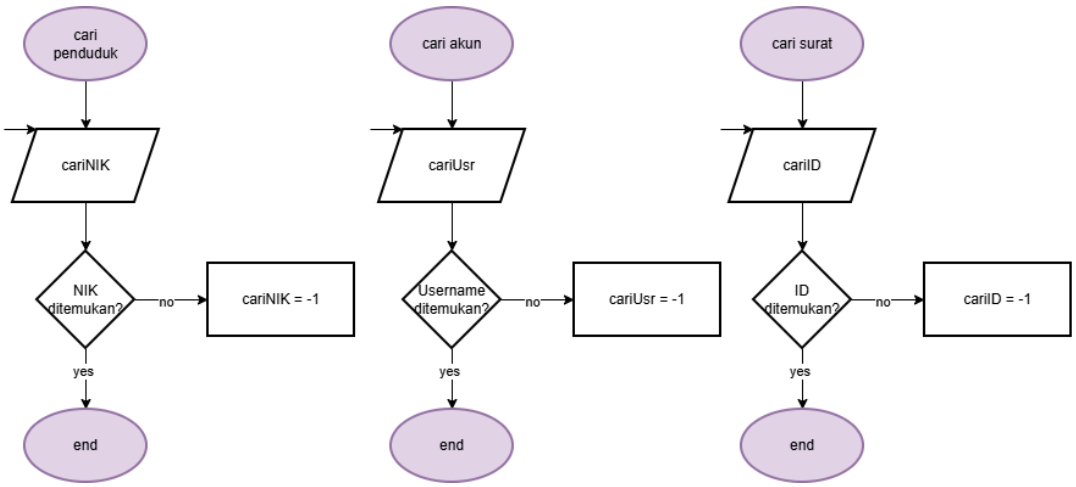
Gambar 1.5 Menu Penduduk

Dalam menu penduduk admin punya 4 pilihan dalam mengatur data penduduk. Melihat daftar penduduk, menambah penduduk, mengedit data penduduk, dan menghapus penduduk.



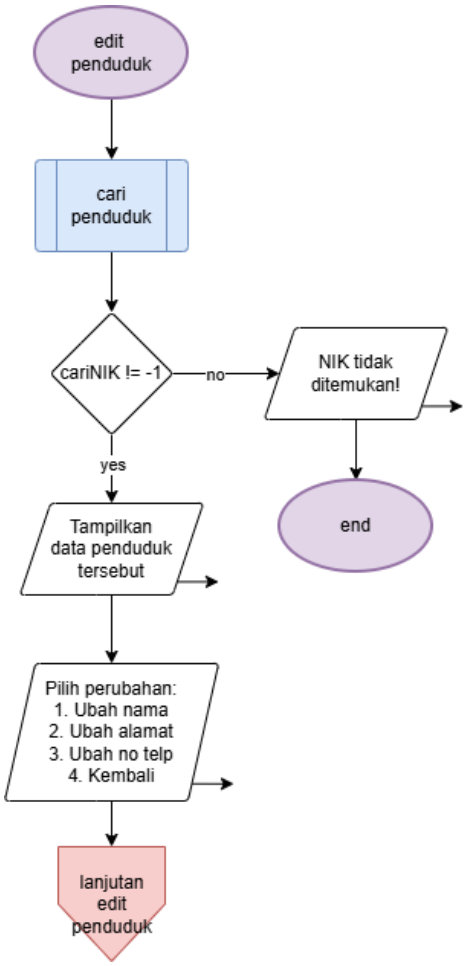
Gambar 1.6 Fungsi Tambah Penduduk

Di fungsi ini, penduduk memiliki 4 *subfield* yaitu NIK sebagai variabel identitas utama, nama, alamat, dan nomor telepon. Dalam menu di atas nomor telepon juga merupakan variabel unik yang tidak bisa sama satu sama lain. Ketika semua *subfield* sudah diisi variabel NIK akan otomatis terisi dengan menggunakan 3 huruf "PDK" ditambah total penduduk saat ini.



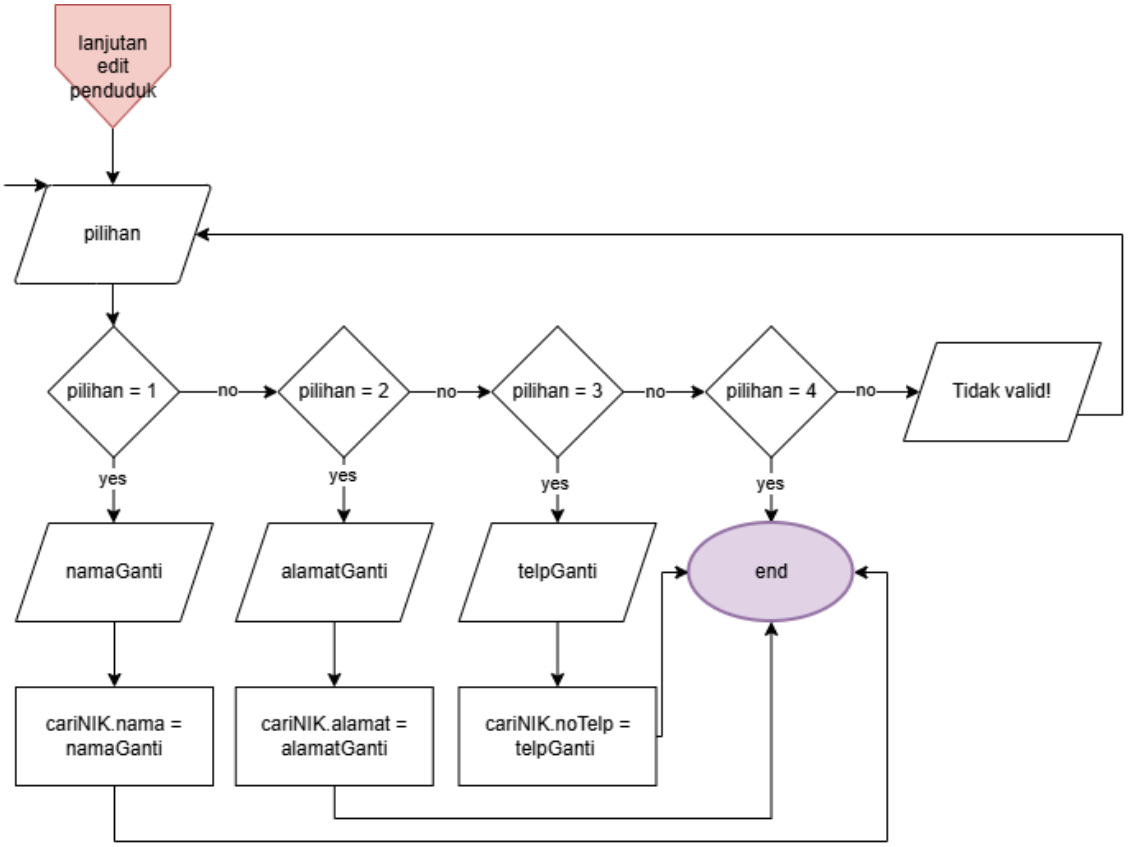
Gambar 1.7 Fungsi Mencari

Ketiga fungsi di atas memiliki alur yang sama. Pengguna memasukkan *field* kunci yang diinginkan. Jika ketemu maka ambil *field* kunci tersebut dan jika tidak maka nilainya menjadi -1 atau tidak ketemu. Fungsi ini digunakan pada proses pengeditan dan penghapusan.



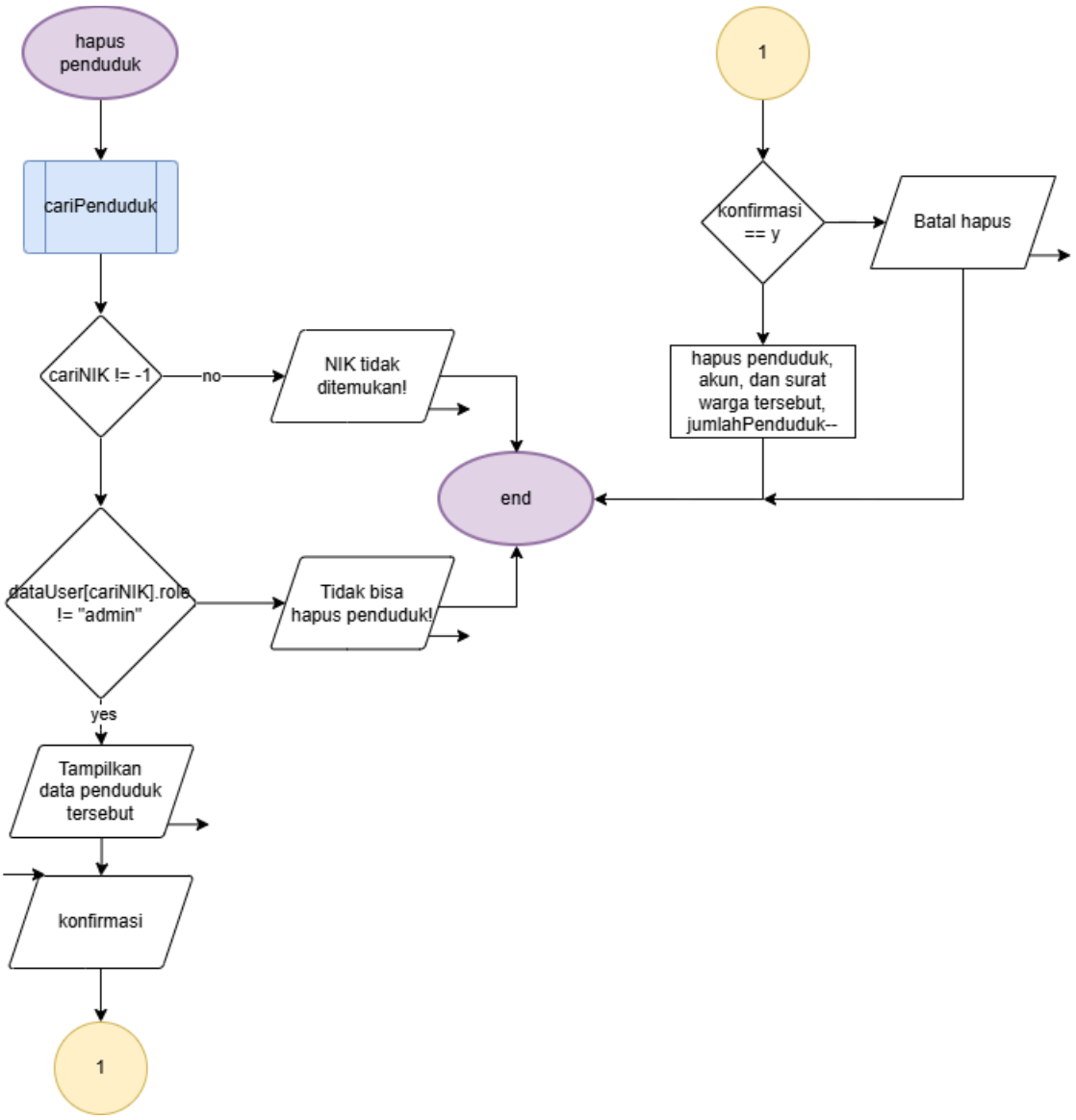
Gambar 1.8 Fungsi Edit Data Penduduk

Di fungsi edit data penduduk, data penduduk yang dicari akan ditampilkan lalu admin bisa memilih apa yang akan diubah.



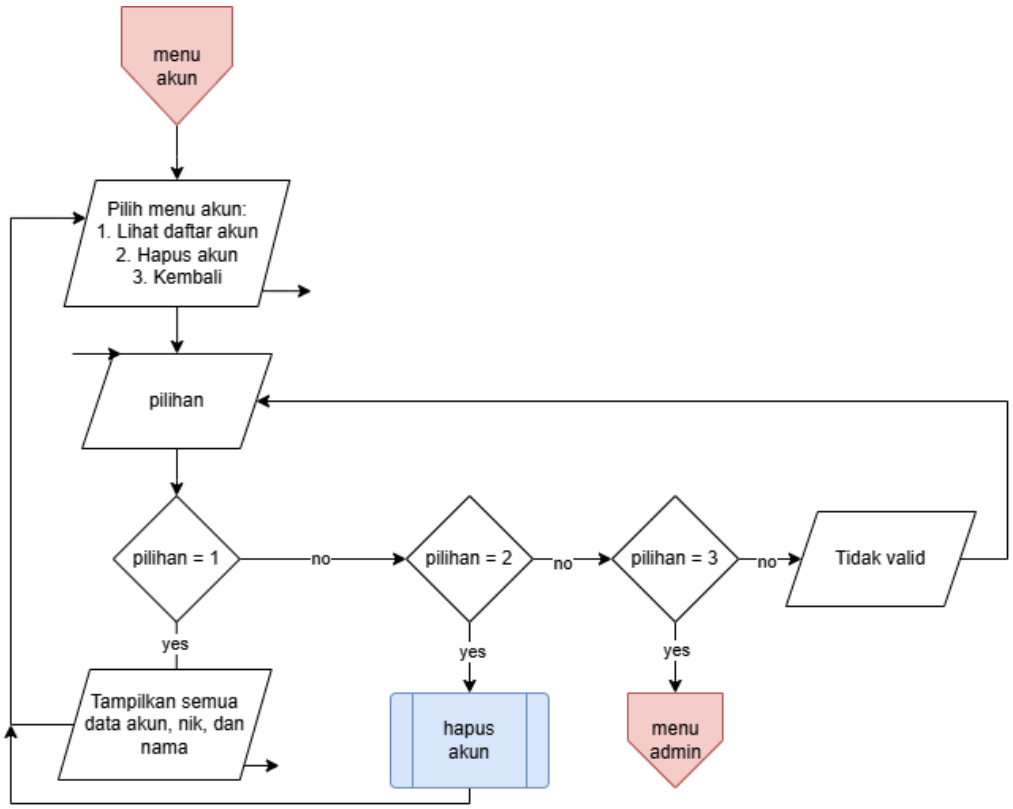
Gambar 1.9 Mengedit Data Penduduk

Admin dapat mengubah 3 *subfield* data penduduk. Prosesnya sama ketika menambah penduduk termasuk nomor telepon unik. Setelah mengganti data lama akan ditimpa dengan data baru.



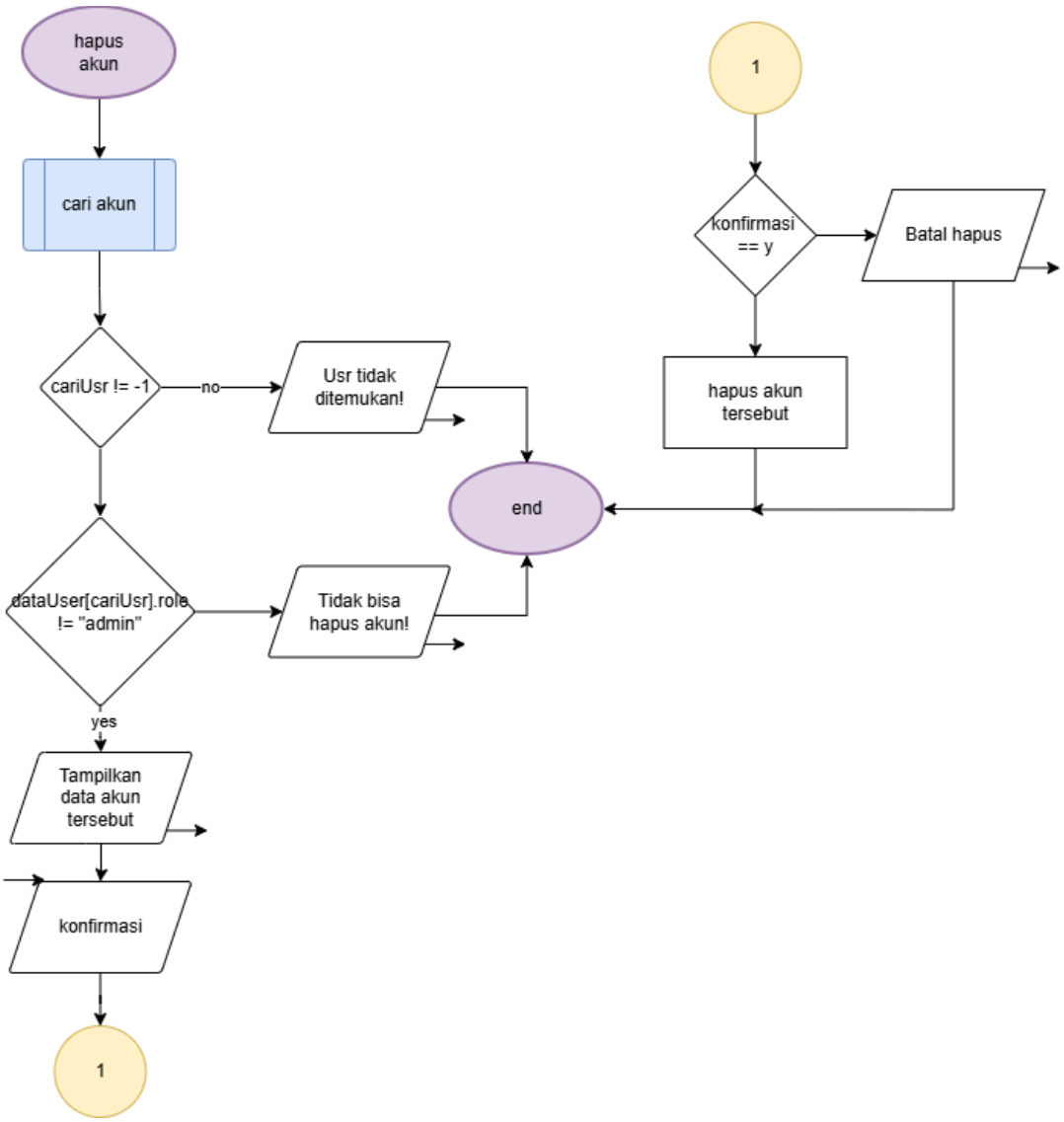
Gambar 1.10 Fungsi Hapus Penduduk

Seperti saat akan mengedit data penduduk, admin juga harus memasukkan NIK yang diinginkan untuk memulai proses penghapusan. Adapun syarat tambahan yaitu admin tidak dapat menghapus akun sesama admin. Admin diminta konfirmasi sebelum betul-betul menghapus data penduduk. Ketika data suatu penduduk dihapus maka akun dan semua surat dari penduduk tersebut juga akan dihapus.



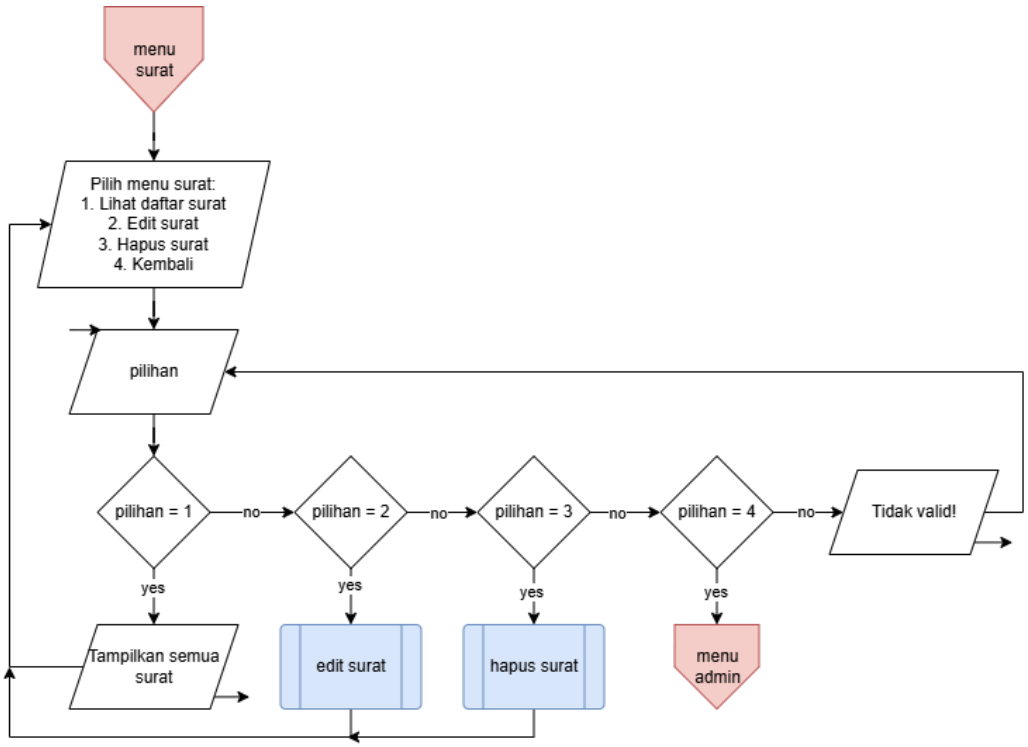
Gambar 1.11 Menu Akun

Setelah selesai di menu *field* penduduk, admin juga bisa memanipulasi akun-akun yang terdaftar. Memilih pilihan 1 akan menampilkan semua akun yang terdaftar beserta *role*, NIK, dan nama.



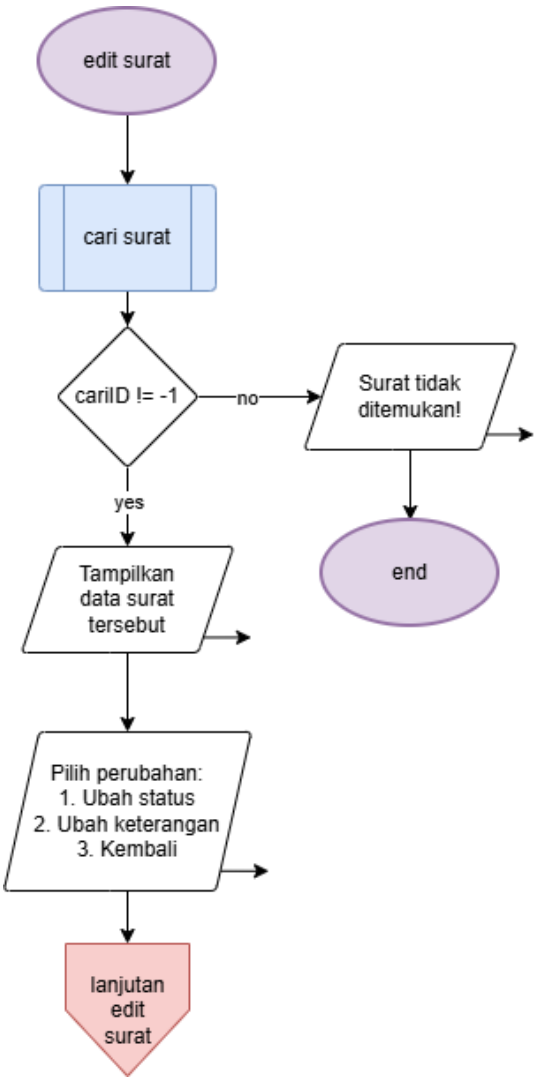
Gambar 1.12 Fungsi Hapus Akun

Fungsi ini berjalan sama seperti hapus penduduk, akun tidak bisa dihapus jika memiliki *role* admin. Setelah konfirmasi barulah akun tersebut dihapus.



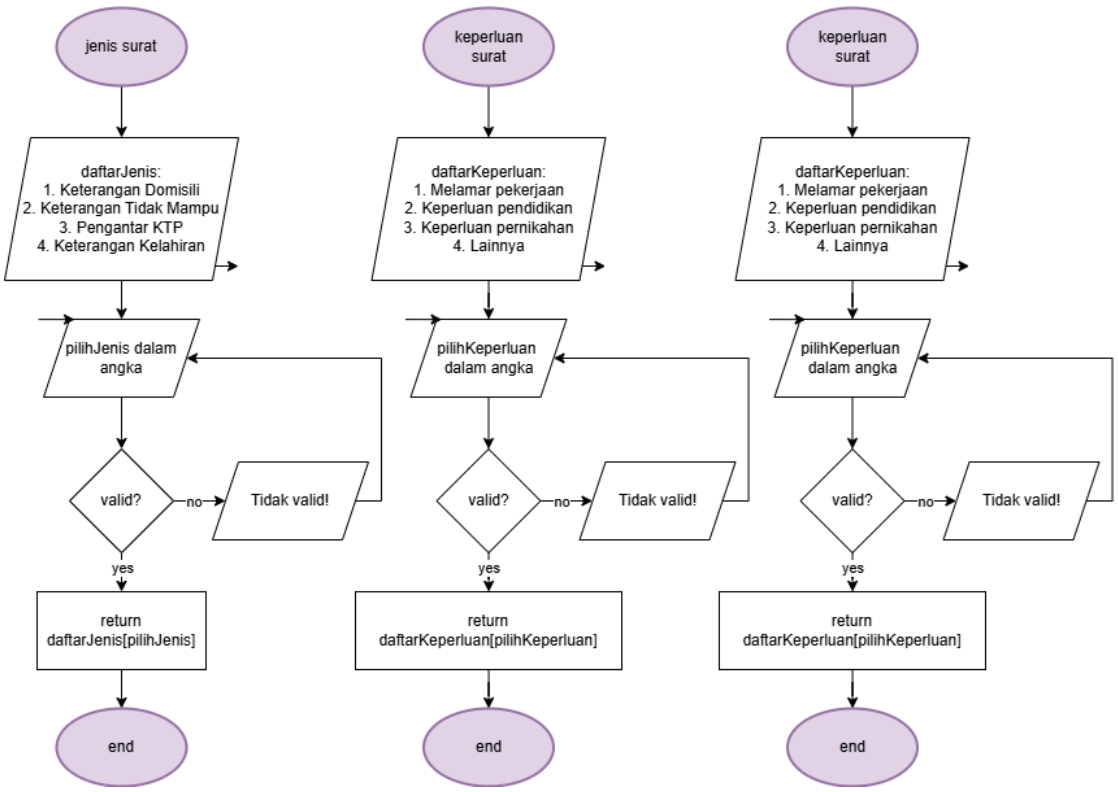
Gambar 1.13 Menu Surat Admin

Pada pilihan ketiga admin dapat mengubah data pada surat-surat ajuan. Data surat punya 6 subfield, yaitu ID surat, NIK pemohon, jenis surat, keperluan, status, keterangan. Admin dapat melihat daftar surat, mengedit surat, dan menghapus surat.



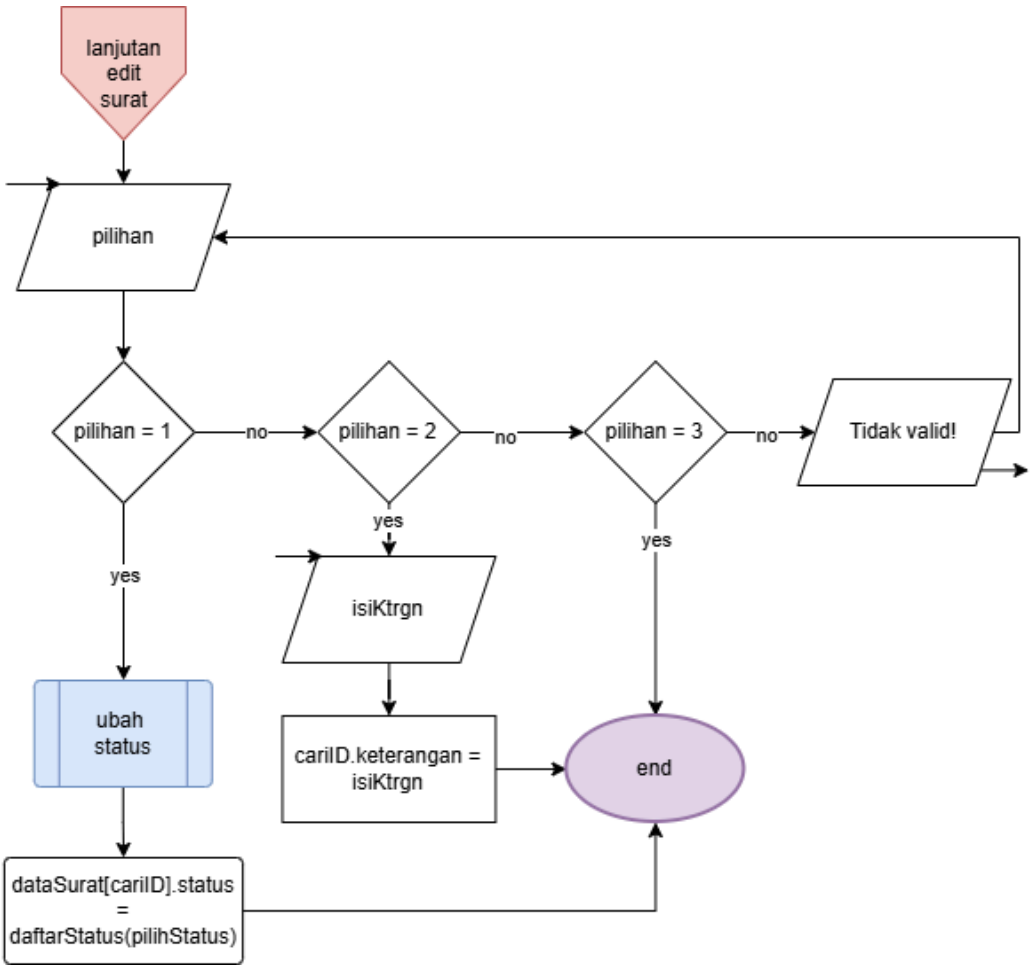
Gambar 1.14 Edit Surat Versi Admin

Panggil fungsi cari surat seperti pada *field* sebelumnya. Lalu pilih perubahan yang diinginkan.



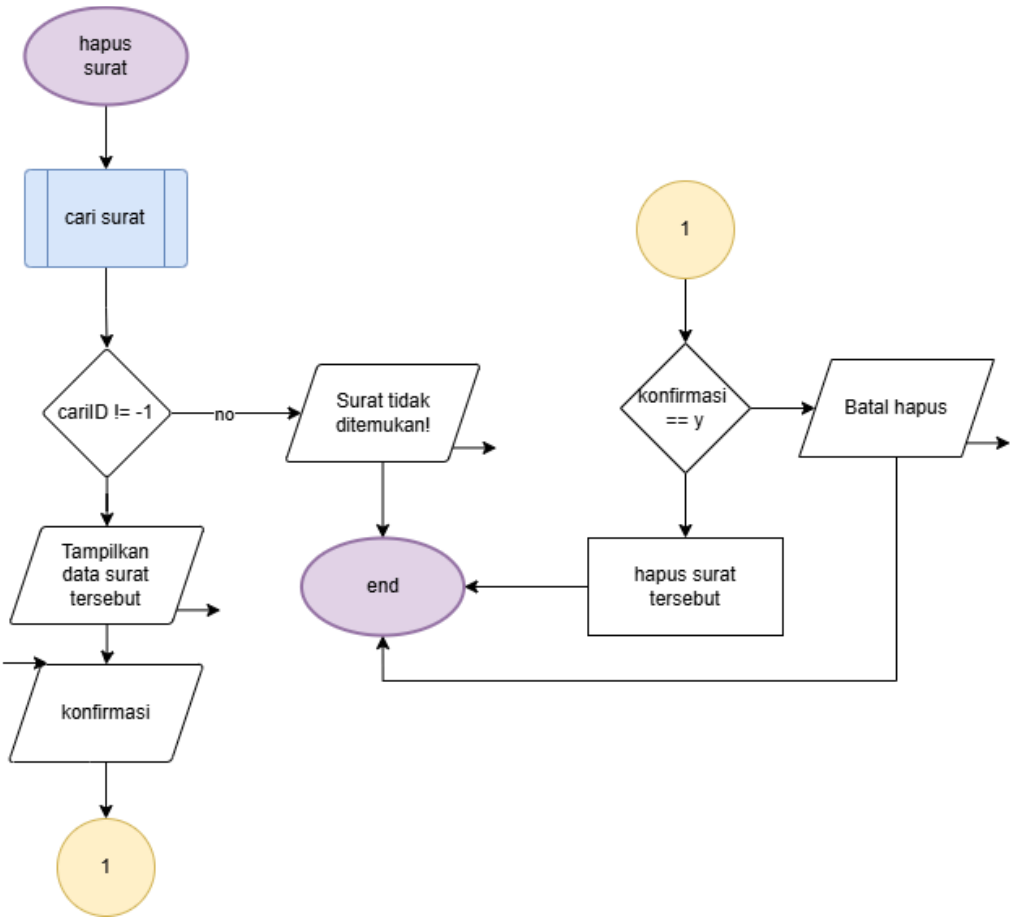
Gambar 1.15 Fungsi Ubah Subfield Surat

Jenis, keperluan, dan status merupakan *subfield* pada surat yang menggunakan fungsi sendiri. Digunakan dalam pengeditan dan pembuatan surat, masing-masing berjalan dengan cara yang sama. Setiap *subfield* memiliki daftar pilihan, kemudian disimpan dalam *integer*.



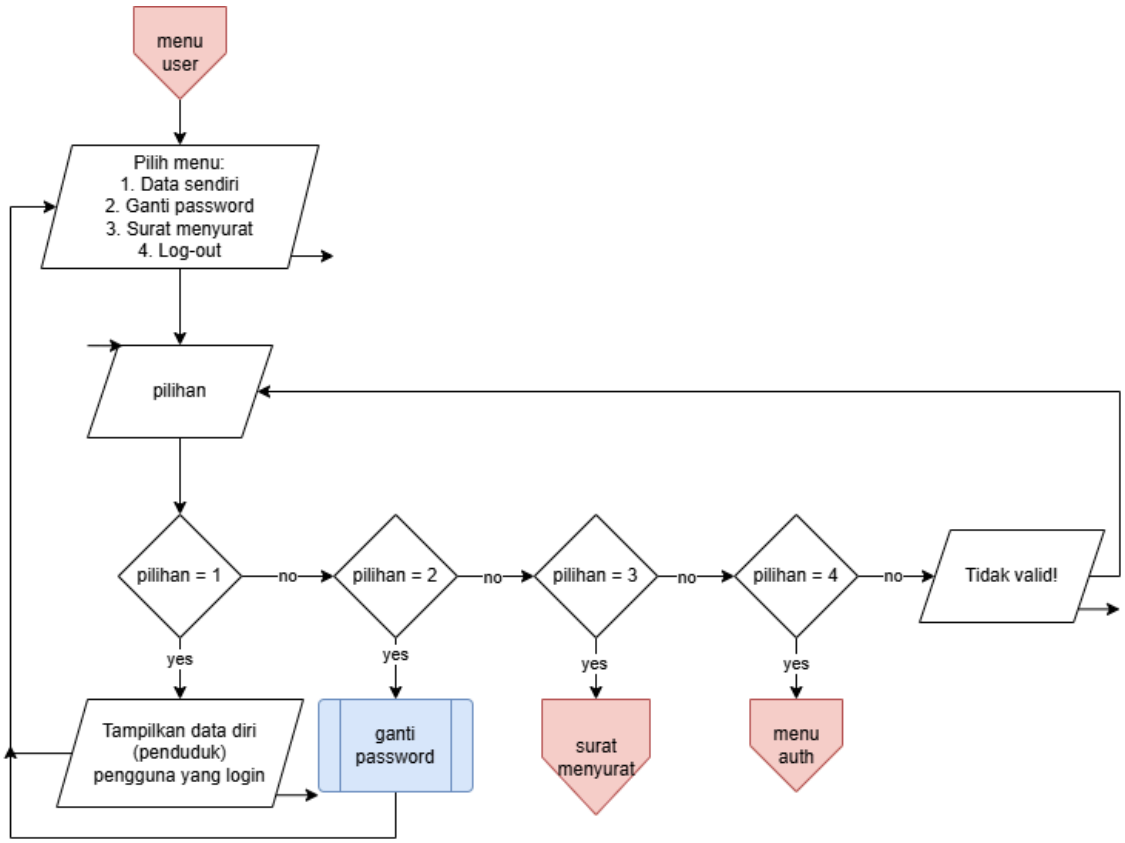
Gambar 1.16 Lanjutan Edit Surat Versi Admin

Admin tidak dapat mengubah data surat sepenuhnya, hanya di bagian status dan keterangan. Status menunjukkan perubahan surat pada waktu ke waktu, sementara keterangan menunjukkan deskripsi surat yang lebih detail kepada pemohon.



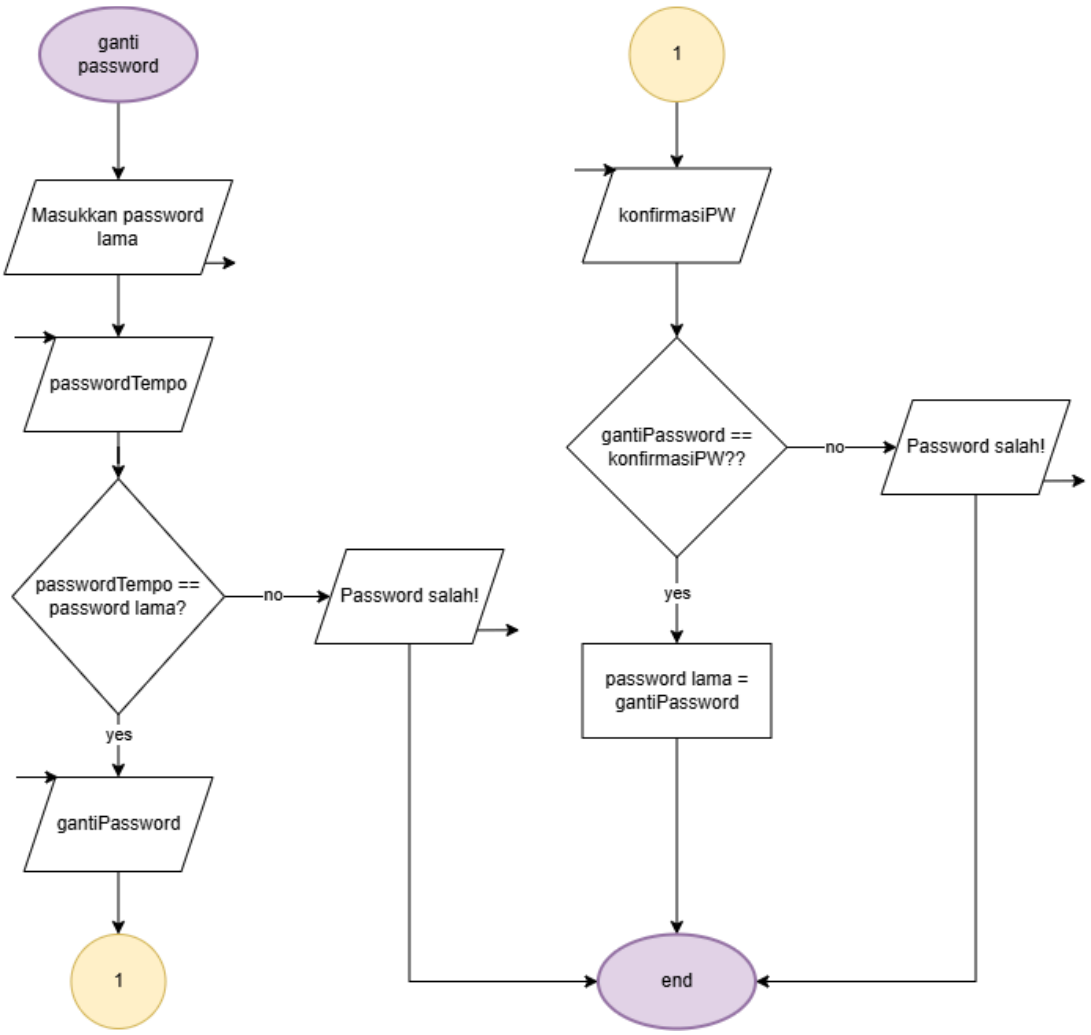
Gambar 1.17 Hapus Surat Versi Admin

Prosesnya sama dengan hapus penduduk dan hapus akun. Masukkan ID surat pada dengan fungsi, meminta konfirmasi, dan hapus surat tersebut.



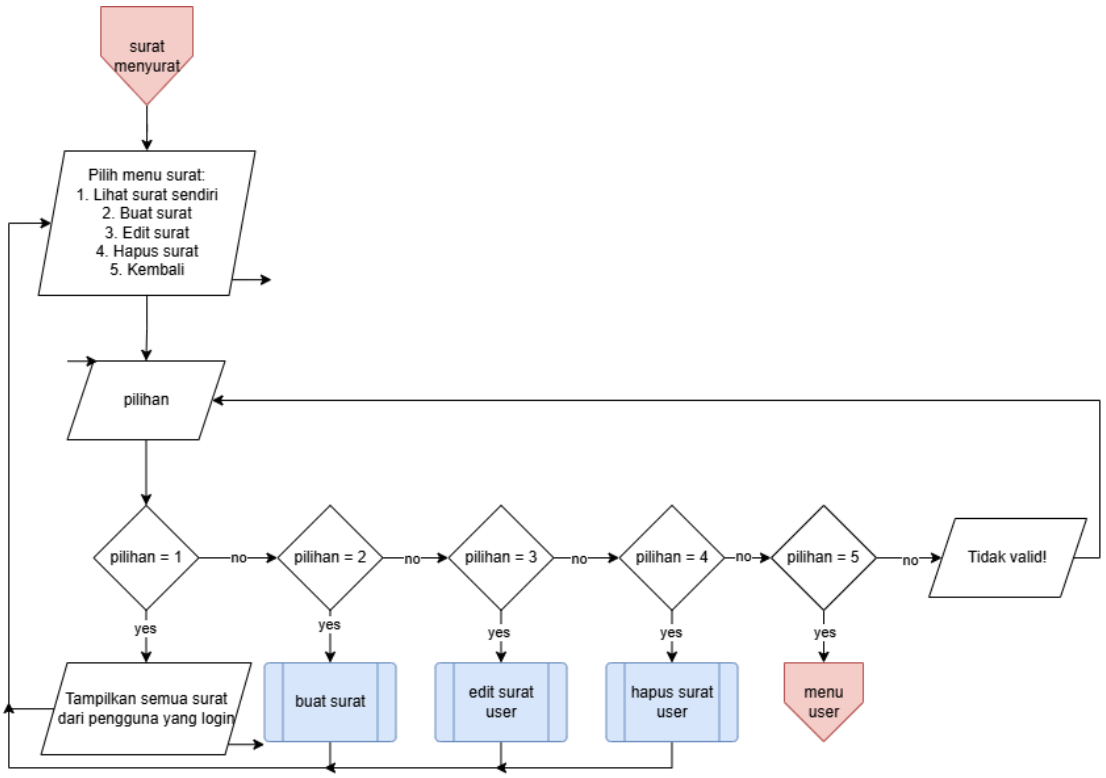
Gambar 1.18 Menu User

Selain admin, ada juga *role user*. *Role* ini hanya bisa memanipulasi data yang terikat dengan dirinya sendiri dan memiliki beberapa batasan. User punya 3 menu utama, menampilkan data mereka, mengganti *password*, dan yang paling utama menu surat menyurat.



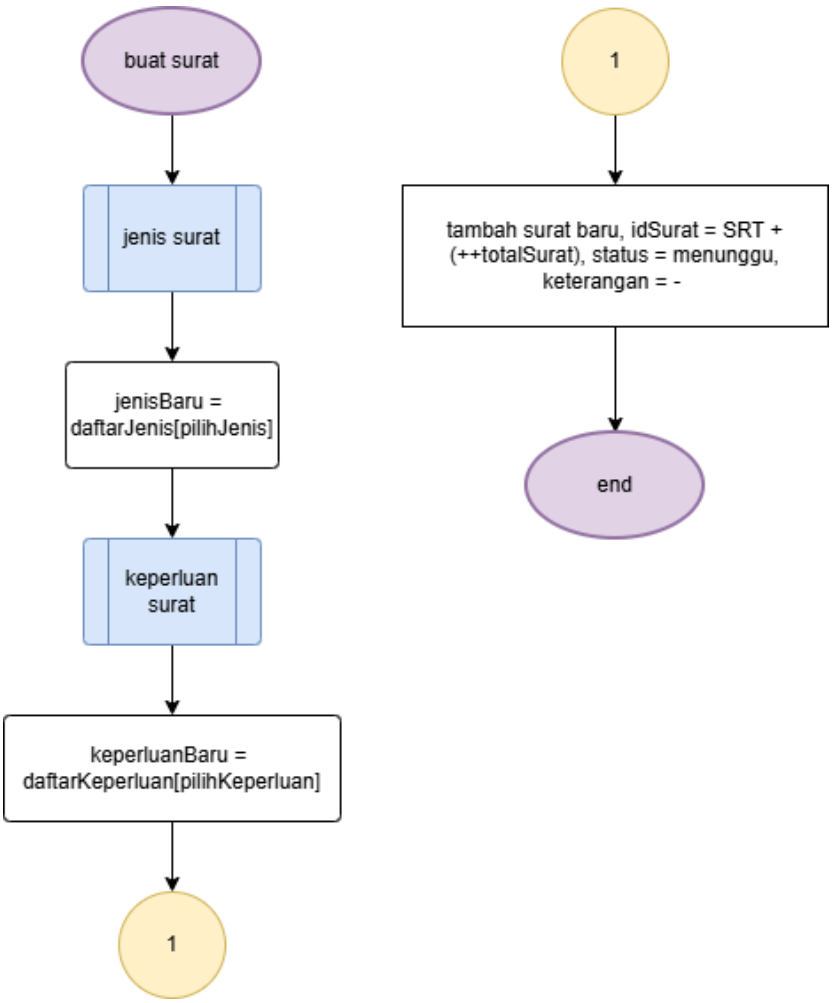
Gambar 1.19 Fungsi Mengganti Password

Untuk menjaga keamanan ketika mengganti *password*, *user* harus memasukkan *password* mereka saat ini. Setelah verifikasi selesai barulah *user* bisa memasukkan *password* baru, adapun *user* juga harus mengkonfirmasi *password* baru mereka dengan memasukkan password yang sama sekali lagi. Jika semua langkah di atas berjalan dengan tepat dan benar, barulah *password* lama akan ditimpa dengan yang baru.



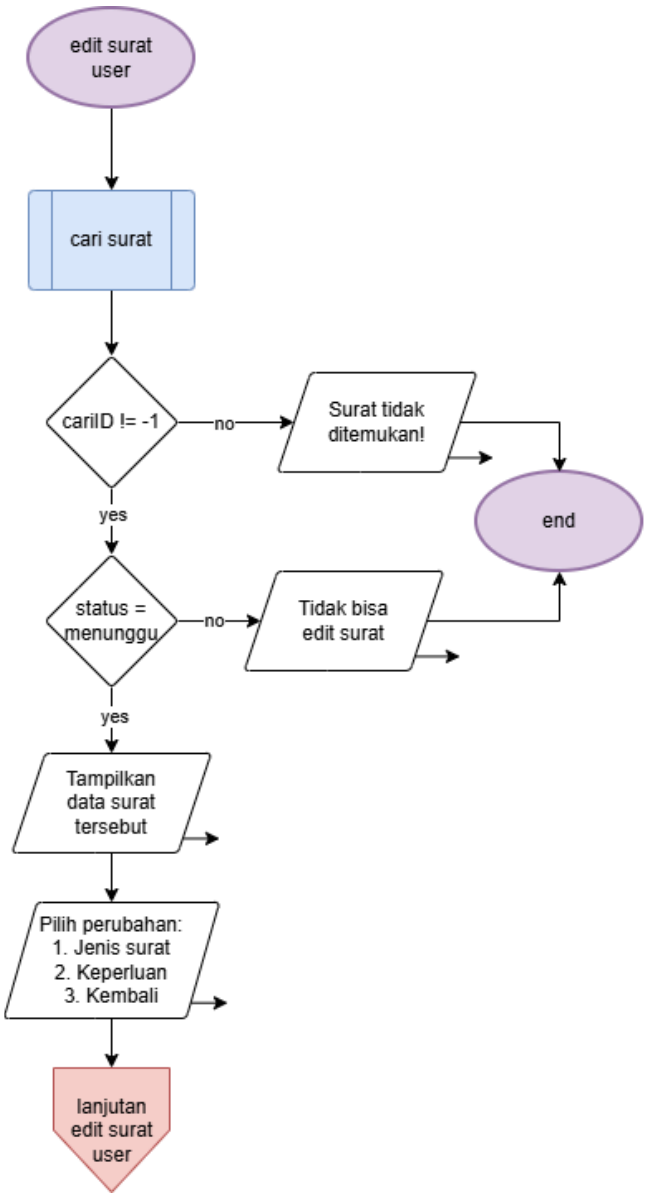
Gambar 1.20 Menu Surat Versi User

Disini menu utama *user*, mereka secara bebas dapat membuat surat, melihat semua surat yang pernah mereka buat/mohon (pilihan 1), mengedit dan menghapus surat mereka.



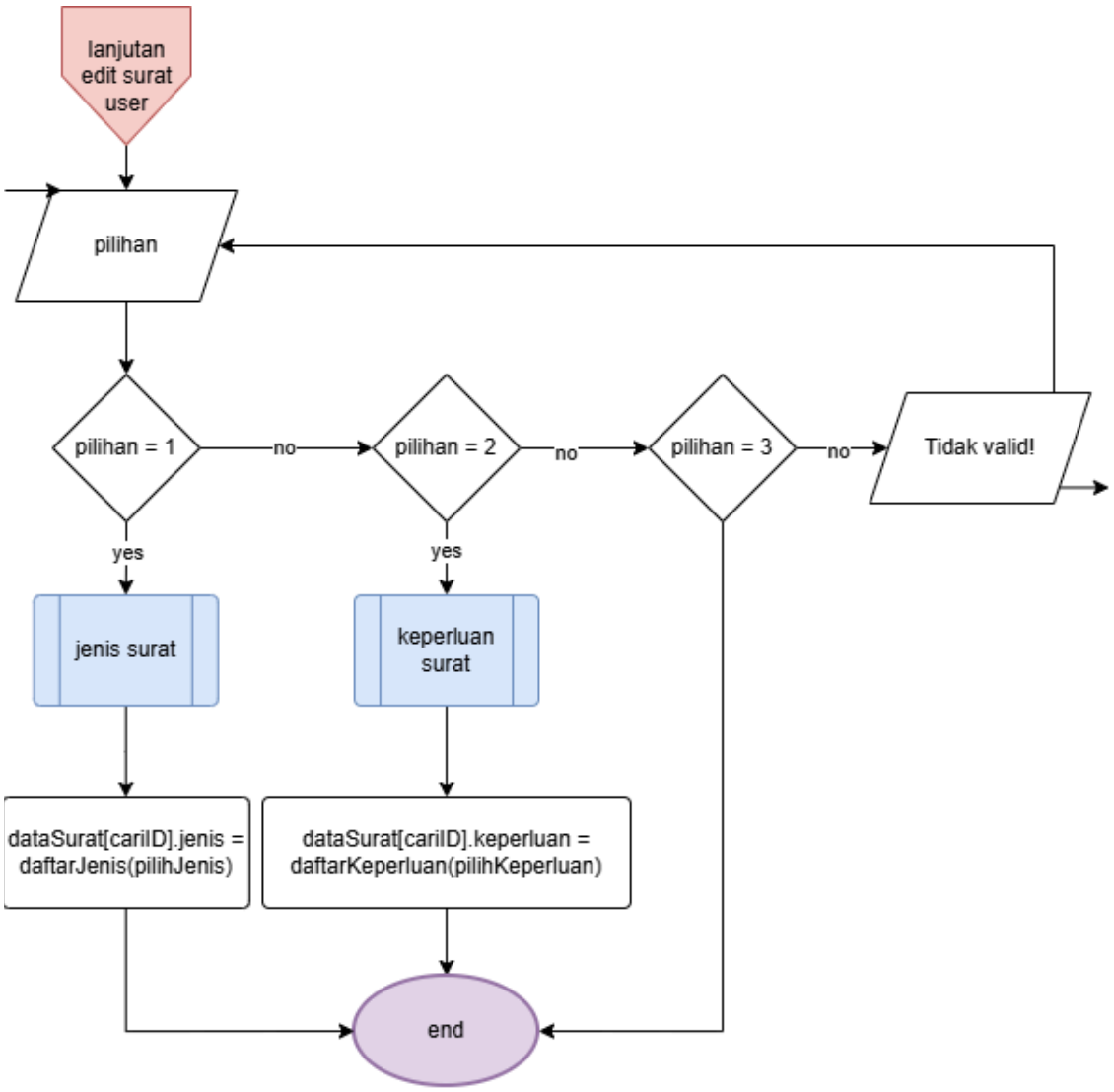
Gambar 1.21 Fungsi Buat Surat

Dalam membuat sura, *user* hanya bisa memilih jenis dan keperluan surat sesuai daftar. Ketika semua sudah selesai, barulah program menambah surat tersebut. ID surat otomatis bertambah seperti NIK, status akan ditetapkan ke menunggu dan isi keterangan kosong yang nantinya akan diubah admin.



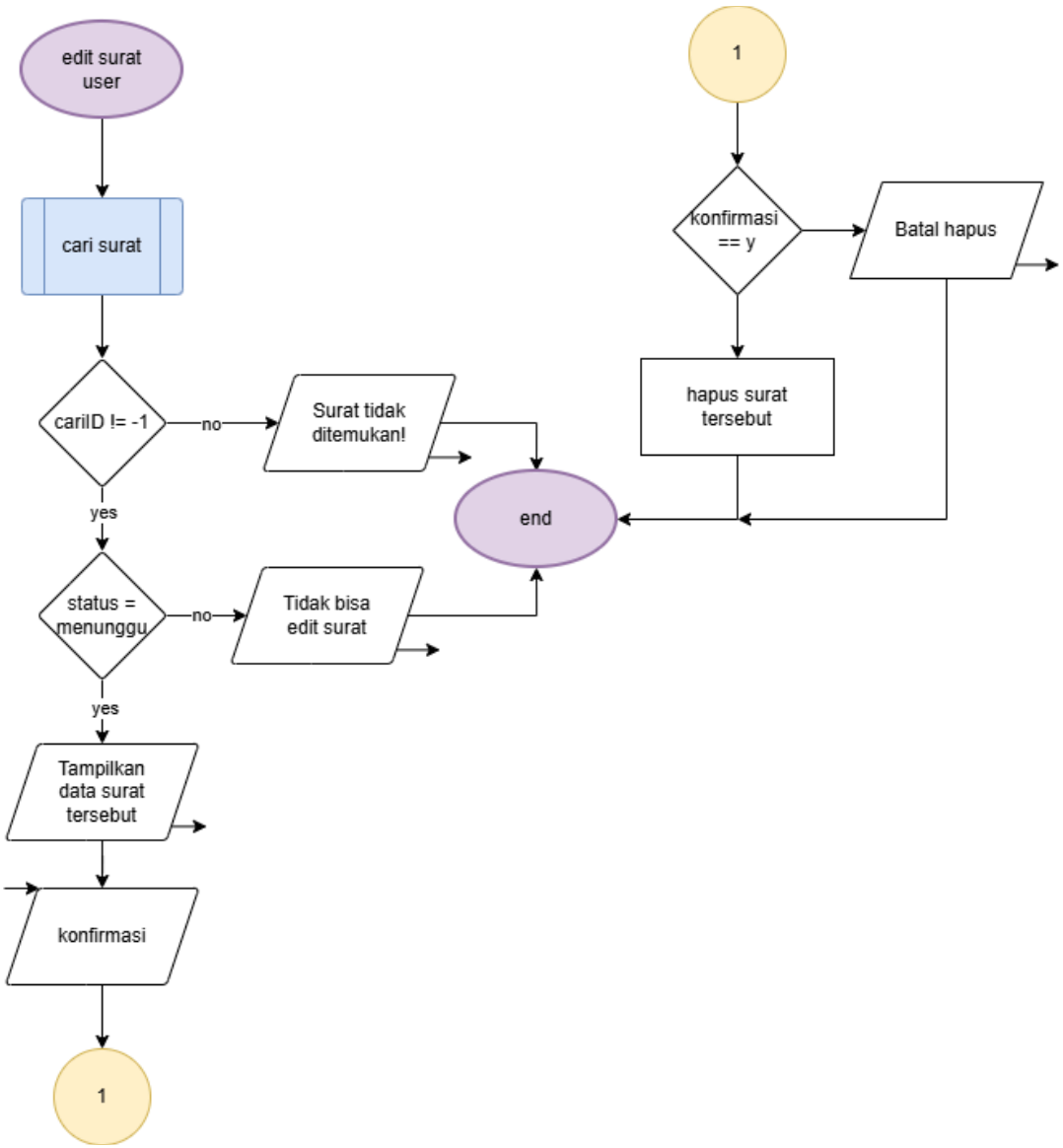
Gambar 1.22 Mengedit Surat Versi User

Sama halnya ketika admin ingin mengedit surat, *user* juga harus memasukkan ID surat yang ingin diubah. Namun disini ada persyaratan, *user* hanya dapat mengedit surat selama status surat tersebut masih menunggu, selebihnya akan ditolak program.



Gambar 1.23 Lanjutan Edit Surat User

User hanya mengedit jenis surat dan keperluan surat menggunakan fungsi pada *Gambar 1.15*. Nilai yang didapat dari fungsi kemudian menimpa data lama.



Gambar 1.24 Hapus Surat Versi User

Proses sama saat mengedit surat, surat hanya bisa dihapus jika status masih menunggu. Setelah *user* mengkonfirmasi penghapusan barulah surat tersebut dihapus sepenuhnya.

2. Deskripsi Singkat Program

Program administrasi balai kota dikelola oleh sekumpulan admin yang memegang database penduduk-penduduknya. Mereka dapat memanipulasi database tersebut. Selain itu mereka juga mengatur akun-akun non-admin. Tujuan utama program ini yaitu kegiatan surat menyurat, user atau pengguna biasa dibebaskan dalam permohonan surat yang ditujukan ke administrasi balai kota. Admin tidak bisa mengubah keaslian surat-surat tersebut melainkan hanya membantu si pemohon dengan terus memperbarui status dan keterangan yang ada pada surat.

3. Source Code

Pada bagian ini alur program dalam *source code* akan dijelaskan, mulai dari variabel, fungsi, hingga fungsi *main*. Penjelasan akan dibagi menjadi beberapa kategori sesuai tipe fungsi dan prosedur yang ada.

3.1 Library dan Variabel Global

```
#include <iostream>
#include <tabulate/table.hpp>
#include <string>
using namespace tabulate;
using namespace std;
struct Penduduk {
    string nik;
    string nama;
    string alamat;
    string noTelp;
};
struct User {
    string usr;
    string pw;
    string role;
    Penduduk dataDiri;
};
struct Surat {
    string idSurat;
    string nikPemohon;
    string jenisSurat;
    string keperluan;
    string status;
    string keterangan;
```

```
};
    string daftarJenis[] {
        "Surat Keterangan Domisili",
        "Surat Keterangan Tidak Mampu",
        "Surat Pengantar KTP",
        "Surat Keterangan Kelahiran"
    };
    string daftarKeperluan[] {
        "Melamar pekerjaan",
        "Keperluan pendidikan",
        "Keperluan pernikahan",
        "Lainnya"
    };
        string daftarStatus[] {
        "menunggu",
        "diproses",
        "selesai",
        "ditolak"
    };
```

Source Code 3.1.1 Daftar Library dan Variabel Global

Sebelum masuk ke program, mari bedah awalan program. Disini kita memiliki 3 *struct* utama yaitu 'Penduduk', 'User' dan 'Surat' yang berisi *field-field* pembangun yang akan digunakan untuk CRUD program. *Struct* 'User' menggunakan *nested struct* pada 'Penduduk' yang disimpan di 'dataDiri'.

Lalu ada 3 *array of string* yang berisi nilai dari 3 *field* 'Surat' yaitu 'jenisSurat', 'keperluan', dan 'status'. Nilai ini memang sudah ditetapkan untuk memudahkan program dan mencegah isi pengajuan surat berisi hal tidak yang perlu. Selain itu program ini menggunakan *library* Tabulate untuk mempercantik tampilan *output* program.

3.2 Daftar Fungsi Minor dan Fungsi Validasi

Fungsi-fungsi di bawah ini bertipekan fungsi validasi yang membantu dalam proses penginputan program. Bertujuan mencegah adanya kesalahan karakter atau input di beberapa bagian.

```
int tampilMenu(string judul, string opsi[], int jumlah) {
    int pilih;
    Table table;
    table.add_row({judul});
    for (int i=0; i < jumlah; i++) {
        table.add_row({to_string(i+1) + ". " + opsi[i]});
    }
    cout << table << "\n";
```

```
    cout << "Pilih: ";
    while (!(cin >> pilih) || pilih < 1 || pilih > jumlah) {
        cin.clear();
        cin.ignore(100, '\n');
        cout << "Tidak valid!\n";
    }
    return pilih;
```

Source Code 3.2.1 Fungsi Menampilkan Daftar Menu

Program ini berisi banyak CRUD yang menyebabkan banyak blok kode untuk menampilkan *output* menu dan pilihan pengguna. Oleh karena itu mari gunakan fungsi 'tampilMenu' ini. Fungsi ini punya 3 parameter, salah satunya ada 'opsi[]', parameter ini berisi *array of string* yang harus dibuat sebelum memanggil fungsi, adapun parameter 'jumlah' merupakan jumlah dari pilihan menu.

Fungsi ini juga menggunakan tabel dari Tabulate, pertama deklarasi tipe data 'pilih' dan panggil tabel dengan 'Table table'. *Add_row* berfungsi untuk menambah baris tabel diikuti parameter 'judul'. Kemudian lakukan perulangan *for* dari *array of struct* yang diargumenkan dan gabung dengan *add_row*. Penggunaan *to_string* untuk menampilkan angka dengan menambahkan indeks saat ini dengan 1.

Perulangan *while* berguna sebagai *error handling* kalau pengguna memasukkan pilihan kurang atau lebih dari jumlah. Ketika pengguna memasukkan input selain angka maka program langsung meminta pengguna input ulang dan notifikasi kesalahan input. Ketika 'cin' gagal membaca ia akan beku dan 'cin.clear()' yang mengembalikannya ke semula. Adapun 'cin.ignore()' akan membuat sisa karakter pada buffer, maks 100 karakter atau saat membaca enter/newline. Terakhir kembalikan nilai 'pilih'.

```
bool konfirmasi(string pesan) {
    char jawaban;
    cout << pesan << " (y/n): ";
    cin >> jawaban;
    return jawaban == 'y';
```

Source Code 3.2.2 Fungsi Meminta Konfirmasi

Fungsi ini meminta konfirmasi dari dua karakter. Digunakan pada permintaan konfirmasi sebelum menghapus suatu data. Karakter 'y' menjadi nilai *true* dari fungsi ini.

```
string inputStr(string pesan) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil != "") return hasil;
```

```
        cout << "Tidak boleh kosong!\n";
```

Source Code 3.2.3 Fungsi Input Kosong

Fungsi ini mencegah pengguna memasukkan inputan kosong. Digunakan di berbagai bagian yang meminta input seperti saat menambah data. Kerjanya bandingkan inputan dengan *string* kosong (“”), jika iya maka tidak kembalikan nilai.

```
string inputMax(string pesan, int maxLen) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil == "") {
            cout << "Tidak boleh kosong!\n";
        } else if (hasil.length() > maxLen) {
            cout << "Tidak boleh lebih dari " << maxLen << " karakter!\n";
        } else {
            return hasil;
        }
    }
}
string inputMin(string pesan, int minLen) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil == "") {
            cout << "Tidak boleh kosong!\n";
        } else if (hasil.length() < minLen) {
            cout << "Tidak boleh kurang dari " << minLen << " karakter!\n";
        } else {
            return hasil;
```

Source Code 3.2.4 Fungsi Jumlah Karakter

Fungsi ini berguna untuk mencegah jumlah karakter dari suatu inputan melebihi atau kurang dari batas yang diberikan. Prosesnya gunakan ‘hasil.length()’ untuk menghitung panjang inputan, bandingkan dengan ‘maxLen’/’minLen’.

```
string hanyaAngka(string pesan) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil == "") {
            cout << "Tidak boleh kosong!\n";
            continue;
        }
```

```
        bool valid = true;
        for (int i = 0; i < hasil.length(); i++) {
            if (!isdigit(hasil[i])) {
                valid = false;
                break;
            }
        }
        if (!valid) {
            cout << "Hanya boleh angka!\n";
        } else {
            return hasil;
```

Source Code 3.2.5 Fungsi Menerima Angka

Fungsi ini digunakan saat meminta inputan nomor telepon, mencegah adanya karakter huruf dan simbol di dalamnya sekaligus mencegah input kosong. Prosesnya menggunakan *flag* 'valid' dan *isdigit* untuk membaca karakter angka di dalamnya.

```
string inputOpsional(string pesan, int maxLen) {
    string hasil;
    while (true) {
        cout << pesan;
        getline(cin, hasil);
        if (hasil.length()>maxLen) {
            cout << "Tidak boleh lebih dari " << maxLen << " karakter!\n";
        } else {
            return hasil;
```

Source Code 3.2.6 Fungsi Input Kosong Opsional

Fungsi ini digunakan saat mengisi *field* keterangan pada *struct* 'Surat'. Karena pada *field* itu nilai kosong tidak dilarang.

```
bool statusMenunggu(Surat data[], int index) {
    return data[index].status == "menunggu";
```

Source Code 3.2.7 Fungsi Status Surat

Fungsi ini bertujuan untuk membaca status dari suatu surat, digunakan dalam proses pengeditan surat dan penghapusan surat.

3.3 Daftar Fungsi Autentikasi

Fungsi-fungsi di bawah ini merupakan fungsi dalam proses autentikasi seperti *login* dan registrasi.

```
bool login(User dataUser[], int jumlahUser, int kesempatan, int &indexUser)
{
    if (kesempatan == 0) {
        cout << "Kesempatan habis!\n";
        return false;
    }
    string username, password;
    cout << "Username: ";
    getline(cin, username);
    cout << "Password: ";
    getline(cin, password);
    for (int i = 0; i < jumlahUser; i++) {
        if (username == dataUser[i].usr && password == dataUser[i].pw) {
            indexUser = i;
            cout << "Login berhasil! Selamat datang, " << username << "!\n";
            return true;
        }
    }
    cout << "Salah! Sisa kesempatan: " << kesempatan - 1 << "\n";
    return login(dataUser, jumlahUser, kesempatan - 1, indexUser);
```

Source Code 3.3.1 Fungsi Login

Fungsi 'login' di atas menggunakan mekanisme rekursif yaitu fungsi yang memanggil dirinya sendiri dan memerlukan suatu kondisi berhenti. Fungsi ini menggunakan *pass by reference*, yaitu parameter dengan simbol & pada parameter 'indexUser', dengan ini setiap perubahan yang terjadi akan mempengaruhi variabel aslinya. Fungsi akan berhenti ketika variabel 'kesempatan' menjadi 0. Setiap kesalahan inputan *username* dan *password*, fungsi kembali dipanggil dengan kesempatan berkurang 1.

Sebelum masuk ke fungsi registrasi, ada beberapa fungsi pendukung yang membantu saat registrasi. Registrasi **hanya bisa dilakukan** jika data penduduk dari pengguna yang mau regis terdaftar dalam data penduduk dan tidak memiliki akun.

```
bool usernameAda (User data[], int jumlah, string username) {
    for (int i=0; i < jumlah; i++) {
        if (data[i].usr == username) return true;
    }
    return false;
```

Source Code 3.3.2 Fungsi Username Terduplikat

Pengguna baru akan ditolak jika memasukkan *username* yang sudah digunakan oleh pengguna lain. Prosesnya lakukan *for* di data 'User' untuk membandingkan inputan *username* dari pengguna yang sedang regis.

```
int cariPendudukDariNama (Penduduk data[], int jumlah, string nama, int
hasilCari[]) {
    int jumlahHasil = 0;
    for (int i=0; i < jumlah; i++) {
        if (data[i].nama == nama) {
            hasilCari[jumlahHasil] = i;
            jumlahHasil++;
        }
    }
    return jumlahHasil;
```

Source Code 3.3.3 Fungsi Cari Nama Penduduk

Bagaimana jika ditemukan beberapa data penduduk dengan nama sama dan semuanya belum memiliki akun? Selain meminta *username* dan *password*, pengguna juga harus memasukkan nama lengkap. Setelah input selesai dilakukan *for* dalam data 'Penduduk'. Setiap ditemukan nama sama, indeks dari penduduk dengan nama tersebut akan disimpan di array 'hasilCari'. Setiap ketemu nama sama 'jumlahHasil' bertambah. Terakhir *array* 'hasilCari' dan 'jumlahHasil' dikembalikan.

```
bool punyaAkun (User dataUser[], int jumlahUser, string nik) {
    for (int i=0; i < jumlahUser; i++) {
        if (dataUser[i].dataDiri.nik == nik) return true;
    }
    return false;
```

Source Code 3.3.4 Fungsi Penduduk Punya Akun

Sebelumnya dijelaskan hanya data penduduk yang belum punya akun yang hanya bisa registrasi. Alurnya lakukan *for* dengan melihat *nested struct* 'dataDiri.nik' dengan NIK yang ingin dibandingkan.

```
void registarsi(User dataUser[], int &jumlahUser, Penduduk dataPenduduk[],
int jumlahPenduduk) {
    string usernameBaru = inputStr("Username: ");
    if (usernameAda(dataUser, jumlahUser, usernameBaru)) {
        cout << "Username sudah digunakan!\n";
        return;
    }

    string passwordBaru = inputMin("Password: ", 3);

    string namaCari = inputStr("Nama lengkap: ");
    int hasilCari[100];
```

```
    int jumlahHasil = cariPendudukDariNama(dataPenduduk, jumlahPenduduk,
namaCari, hasilCari);

    int filter[100];
    int jumlahFilter = 0;
    for (int i=0; i < jumlahHasil; i++) {
        if (!punyaAkun(dataUser, jumlahUser,
dataPenduduk[hasilCari[i]].nik)) {
            filter[jumlahFilter++] = hasilCari[i];
        }
    }

    if (jumlahFilter == 0) {
        cout << "Nama tidak ditemukan atau nama sudah punya akun!\n";
        return;
    }

    int indexPenduduk;
    if (jumlahFilter == 1) {
        indexPenduduk = filter[0];
    } else {
        Table table;
        table.add_row({"No", "Nama", "Alamat", "NIK"});
        for (int i = 0; i < jumlahFilter; i++) {
            table.add_row({
                to_string(i+1),
                dataPenduduk[filter[i]].nama,
                dataPenduduk[filter[i]].alamat,
                dataPenduduk[filter[i]].nik
            });
        }
        table.column(0).format().width(4); // No
        table.column(1).format().width(16); //Nama
        table.column(2).format().width(16); // Alamat
        table.column(3).format().width(6); // NIK
        cout << table << "\n";

        int pilih;
        cout << "Pilih nomor: ";
        while (!(cin >> pilih) || pilih < 1 || pilih > jumlahFilter) {
            cin.clear();
            cin.ignore(100, '\n');
            cout << "Tidak valid!\n Pilih: ";
        }
        indexPenduduk = filter[pilih-1];
    }

    dataUser[jumlahUser++] = {usernameBaru, passwordBaru, "user",
```

```
dataPenduduk[indexPenduduk]};
    cout << "Registrasi berhasil!\n";
    cout << "NIK kamu: " << dataPenduduk[indexPenduduk].nik << "\n";
```

Source Code 3.3.5 Prosedur Registrasi

Kita masuk pada prosedur registrasi. Jika fungsi 'usernameAda' *true* maka pengguna kembali ke menu autentikasi. Jika tidak lanjut masukkan *password*, syaratnya tidak boleh kurang dari 3 karakter dan masukkan nama lengkap.

Panggil fungsi 'cariPendudukDariNama' untuk menyimpan setiap nama sama ke *array* 'hasilCari'. Lanjut ke bagian filter mereka yang belum punya akun. Perulangan dilakukan di *array* 'hasil', panggil fungsi 'adaAkun' yang setiap kali mengembalikan nilai *false*, maka data penduduk tersebut masuk ke *array* 'filter', 'jumlahFilter' bertambah satu setiap nilai yang ditemukan.

Jika tidak ditemukan maka artinya nama tidak ditemukan atau semua penduduk dengan nama tersebut sudah memiliki akun. Jika hanya menemukan satu data maka bisa langsung disimpan di variabel 'indexPenduduk'. Jika ditemukan data lebih dari satu, semua data ditampilkan dalam tabel beserta *field* lainnya. Pengguna harus memilih data yang sesuai. Terakhir akun baru dibuat dengan *role* bawaan yaitu *user* dan *nested struct* dari 'dataPenduduk[indexPenduduk]'.

3.4 Daftar Fungsi Pendukung Fitur Utama

```
void tampilSuratByNIK(Surat data[], int jumlah, string nik) {
    Table table;
    table.add_row({"No", "ID Surat", "Jenis Surat", "Keperluan", "Status",
"Keterangan"});
    int no = 1;
    bool ada = false;
    for (int i = 0; i < jumlah; i++) {
        if (data[i].nikPemohon == nik) {
            table.add_row({
                to_string(no++),
                data[i].idSurat,
                data[i].jenisSurat,
                data[i].keperluan,
                data[i].status,
                data[i].keterangan
            });
            ada = true;
        }
    }
```

```
    if (!ada) {
        cout << "Belum ada surat!\n";
        return;
    }
    table.column(0).format().width(4);
    table.column(1).format().width(6);
    table.column(2).format().width(17);
    table.column(3).format().width(11);
    table.column(4).format().width(8);
    table.column(5).format().width(20);
    cout << table << "\n";
```

Source Code 3.4.1 Prosedur Menampilkan Surat Individu

Prosedur ini berbeda dengan prosedur 'tampilSurat' sebelumnya. Prosedur ini berkhusus untuk akun dengan *role user* yang hanya bisa melihat surat mereka sendiri. Perulangan *for* diikuti dengan *if* yang membandingkan 'nikPemohon' dari *struct* 'Surat' dengan NIK dari pengguna yang sedang *login*. Maka semua surat dengan NIK sesuai akan ditampilkan. Adapun *flag* 'ada' berfungsi untuk mengembalikan pengguna ke menu sebelumnya jika sedang tidak memiliki surat. Selebihnya prosedur ini memiliki proses yang sama dengan prosedur 'tampilSurat'.

```
int cari(Penduduk data[], int jumlah, string nik) {
    for (int i=0; i < jumlah; i++) {
        if (data[i].nik == nik) {
            return i;
        }
    }
    return -1;
}
int cari(User data[], int jumlah, string username) {
    for (int i=0; i < jumlah; i++) {
        if (data[i].usr == username) {
            return i;
        }
    }
    return -1;
}int cari(Surat data[], int jumlah, string idSurat) {
    for (int i=0; i < jumlah; i++) {
        if (data[i].idSurat == idSurat) {
            return i;
        }
    }
    return -1;
```

Source Code 3.4.2 Fungsi Overloading Cari

Fungsi *overloading* adalah teknik yang memungkinkan kita membuat beberapa fungsi

dengan nama yang sama, tetapi dengan jumlah atau tipe parameter yang berbeda. Disini terdapat 3 fungsi dengan nama 'cari', namun *struct array* dari parameter pertama di setiap fungsi berbeda tergantung dari apa yang dicari. Setiap fungsi tersebut memiliki proses yang sama. Pengguna memasukkan *field* kunci yang ingin dicari, lakukan perulangan *for* dan bandingkan *field* kunci dengan *field* yang dicari. Kembalikan nilai indeks *array* yang diperoleh yang akan disimpan pada variabel index nantinya. Jika tidak ketemu akan mengembalikan nilai -1.

```
int cariSuratUser(Surat data[], int jumlah, string idSurat, string nik) {
    for (int i=0; i < jumlah; i++) {
        if (data[i].idSurat == idSurat && data[i].nikPemohon == nik) {
            return i;
        }
    }
    return -1;
```

Source Code 3.4.3 Fungsi Mencari Surat Versi User

Fungsi bekerja persis seperti fungsi 'cari' pada fungsi sebelumnya, hanya saja perbandingan juga dilakukan melalui 'nikPemohon' dan NIK dari pengguna yang sedang *login*. Meskipun ID surat itu unik fungsi ini memastikan bahwa setiap pengguna tidak dapat mencari surat dari pengguna lain.

```
void hapus(Penduduk data[], int &jumlah, int index) {
    for (int i= index; i < jumlah - 1; i++) {
        data[i] = data[i+1];
    }
    jumlah--;
}
void hapus(User data[], int &jumlah, int index) {
    for (int i= index; i < jumlah - 1; i++) {
        data[i] = data[i+1];
    }
    jumlah--;
}
void hapus(Surat data[], int &jumlah, int index) {
    for (int i= index; i < jumlah - 1; i++) {
        data[i] = data[i+1];
    }
    jumlah--;
```

Source Code 3.4.4 Prosedur Overloading Hapus

Tiga prosedur ini juga menggunakan mekanisme *overloading*. Prosedur 'hapus' adalah prosedur untuk menghapus suatu data. Parameter 'int &jumlah' menggunakan *address-of operator* '&' agar perubahan nilai jumlah di dalam fungsi langsung

mempengaruhi variabel aslinya di fungsi *main*. Pergeseran dilakukan menggunakan perulangan *for*, indeks dimulai dari indeks *index* kenapa? Kita tidak perlu memulai dari indeks 0 karena indeks yang tepat sudah ditemukan. Jika diperhatikan kondisi perulangan ini adalah jumlah - 1, ini bertujuan untuk mencegah *out of bounds*. Bayangkan pergeseran elemen yaitu elemen kanan mengisi elemen kiri, berarti elemen terakhir tidak ada pengisi jika tidak ada -1. Lalu data ke-i ditimpa dengan data ke-i+1. Terakhir jumlah berkurang, dan karena jumlah dikirim by reference maka pengurangan ini langsung tercermin pada variabel jumlah yang ada di *main*.

3.5 Daftar Fungsi Fitur Utama Admin

```
void tampilPenduduk(Penduduk * data, int jumlah) {
    Table table;
    table.add_row({"No", "NIK", "Nama", "Alamat", "No Telp"});
    for (int i=0; i < jumlah; i++) {
        table.add_row({
            to_string(i+1),
            (data+i)->nik,
            (data+i)->nama,
            (data+i)->alamat,
            (data+i)->noTelp});
    }
    table.column(0).format().width(4); // No
    table.column(1).format().width(6); // NIK
    table.column(2).format().width(16); // Nama
    table.column(3).format().width(16); // Alamat
    table.column(4).format().width(13); // No Telp

    cout << table << "\n";
```

Source Code 3.5.1 Prosedur Menampilkan Daftar Penduduk

Prosedur ini berisi proses menampilkan daftar penduduk. Pakai parameter *pointer* 'Penduduk * data' yang menunjuk ke elemen pertama *array struct* 'Penduduk'. Lakukan perulangan *for* dan gunakan '(data+i)->field' untuk mengakses setiap elemen, di mana (data+i) adalah pointer *arithmetic* untuk berpindah ke elemen ke-i, dan '->' adalah *dereference operator* untuk mengakses *field* dari *struct* yang ditunjuk. Hasilnya ditampilkan menggunakan tabel dengan 5 kolom. Kegunaan format dan *width* pada bagian bawah fungsi merupakan batas jumlah karakter pada satu baris. Jika melebihi maka lanjut ke baris berikutnya. Ini untuk mencegah tabel menjadi sangat panjang.

```
void tampilAkun(User * data, int jumlah) {
    Table table;
    table.add_row({"No", "Username", "Role", "NIK", "Nama"});
    for (int i=0; i < jumlah; i++) {
        table.add_row({
            to_string(i+1),
            (data+i)->usr,
            (data+i)->role,
            (data+i)->dataDiri.nik,
            (data+i)->dataDiri.nama});
    }
    table.column(0).format().width(4); // No
    table.column(1).format().width(16); // Username
    table.column(2).format().width(5); // Role
    table.column(3).format().width(6); // NIK
    table.column(4).format().width(16); // Nama
    cout << table << "\n";
}

void tampilSurat(Surat * data, int jumlah) {
    Table table;
    table.add_row({"No", "ID Surat", "NIK Pemohon", "Jenis Surat",
        "Keperluan", "Status", "Keterangan"});
    for (int i=0; i < jumlah; i++) {
        table.add_row({
            to_string(i+1),
            (data+i)->idSurat,
            (data+i)->nikPemohon,
            (data+i)->jenisSurat,
            (data+i)->keperluan,
            (data+i)->status,
            (data+i)->keterangan});
    }
    table.column(0).format().width(4); // No
    table.column(1).format().width(6); // ID Surat
    table.column(2).format().width(6); // NIK Pemohon
    table.column(3).format().width(17); // Jenis Surat
    table.column(4).format().width(11); // Keperluan
    table.column(5).format().width(8); // Status
    table.column(6).format().width(20); // Keterangan
    cout << table << "\n";
```

Source Code 3.5.2 Prosedur Menampilkan Daftar Akun dan Surat

Kedua prosedur ini memiliki proses yang sama dengan prosedur ‘tampilPenduduk’. Gunakan *deference operator* untuk menampilkan daftar akun dan surat.

```
void tambahPenduduk(Penduduk data[], int &jumlah, int &totalPenduduk) {
    string namaBaru, alamatBaru, noTelpBaru;
```

```
    namaBaru = inputStr("Nama penduduk: ");
    alamatBaru = inputStr("Alamat penduduk: ");
    noTelpBaru = hanyaAngka("No telepon penduduk: ");
    if (telpAda(data, jumlah, noTelpBaru )) {
        cout << "No telepon sudah digunakan!\n";
        return;
    }
    string nikBaru = "PDK" + to_string(++totalPenduduk);
    data[jumlah++] = {nikBaru, namaBaru, alamatBaru, noTelpBaru};
    cout << "Berhasil ditambahkan!\n";
```

Source Code 3.5.3 Prosedur Menambah Penduduk

Simpan semua *field* pada variabel baru, gunakan fungsi 'inputStr' untuk mencegah nilai kosong dan 'hanyaAngka' pada nomor telepon. Lalu validasi menggunakan fungsi 'telpAda'. NIK dari penduduk baru akan otomatis terisi dengan menambah 'totalPenduduk' dengan format "PDK" + "++totalPenduduk" yang diubah ke dalam string. Perbedaan antara jumlah dan total adalah jumlah berisi kuantitas saat ini, sedangkan total adalah kuantitas yang pernah ada, termasuk yang sudah dihapus, ini memungkinkan NIK menjadi *field* unik yang tidak bisa sama satu sama lain. Simpan di data penduduk baru.

```
void editPenduduk(Penduduk dataPenduduk[], int jumlahPenduduk, User
dataUser[], int jumlahUser) {
    string cariNIK;
    cout << "Masukkan NIK dari data penduduk yang ingin diubah: ";
    getline(cin, cariNIK);
    int cariIndex = cari(dataPenduduk, jumlahPenduduk, cariNIK);
    if (cariIndex == -1) {
        cout << "NIK tidak ditemukan!\n";
        return;
    }
    cout << "Data ditemukan: \n";
    Table table;
    table.add_row({"NIK", "Nama", "Alamat", "No Telp"});
    table.add_row({
    dataPenduduk[cariIndex].nik,
    dataPenduduk[cariIndex].nama,
    dataPenduduk[cariIndex].alamat,
    dataPenduduk[cariIndex].noTelp
    });
    table.column(0).format().width(6); // NIK
    table.column(1).format().width(16); // Nama
    table.column(2).format().width(16); // Alamat
    table.column(3).format().width(13); // No Telp
    cout << table << "\n";
    string opsiUbahPenduduk[] = {"Ubah nama penduduk", "Ubah alamat
penduduk", "Ubah no telepon penduduk", "Kembali"};
```

```
    int pilihan = tampilMenu("Pilih perubahan\n", opsiUbahPenduduk, 4 );
    cin.ignore();
    switch (pilihan) {
        case 1: {
            string namaGanti = inputStr("Nama baru: ");
            dataPenduduk[cariIndex].nama = namaGanti;
            for (int i=0; i < jumlahUser; i++) {
                if (dataUser[i].dataDiri.nik == dataPenduduk[cariIndex].nik)
{
                    dataUser[i].dataDiri.nama = namaGanti;
                    break;
                }
            }
            cout << "Berhasil diubah!\n";
            break;
```

Source Code 3.5.4 Prosedur Mengedit Data Penduduk

Prosedur dimulai dengan mencari NIK, dengan fungsi 'cari' versi penduduk. Jika ketemu data penduduk akan ditampilkan dalam tabel dan pengguna pilih perubahan yang diinginkan. Contoh pengguna ingin mengubah nama maka masukkan nama baru kemudian data lama akan ditimpa dengan yang baru. Proses serupa ketika saat menambah penduduk hanya berbeda di kemana data tersebut akan pergi yaitu menimpa data lama. Data pada *struct* 'User' juga diganti mengingat menggunakan *nested struct*. Proses ini juga sama saat mengubah alamat dan nomor telepon.

```
dataPenduduk[cariIndex].alamat = alamatGanti;
if (dataUser[i].dataDiri.nik == dataPenduduk[cariIndex].nik) {
                    dataUser[i].dataDiri.alamat = alamatGanti;
dataPenduduk[cariIndex].noTelp = noTelpGanti;
if (dataUser[i].dataDiri.nik == dataPenduduk[cariIndex].nik) {
                    dataUser[i].dataDiri.noTelp = noTelpGanti;
```

Source Code 3.5.5 Pengeditan Alamat dan Nomor Telepon

```
void hapusPendudukAdmin(Penduduk dataPenduduk[], int &jumlahPenduduk, User
dataUser[], int &jumlahUser, Surat dataSurat[], int &jumlahSurat, int
&indexUser) {
    string cariNIK;
    cout << "Masukkan NIK dari data penduduk yang ingin diubah: ";
    getline(cin, cariNIK);
    int cariIndex = cari(dataPenduduk, jumlahPenduduk, cariNIK);
    if (cariIndex == -1) {
        cout << "NIK tidak ditemukan!\n";
        return;
    }
    bool adaAdmin = false;
```

```
    for (int i = 0; i < jumlahUser; i++) {
        if (dataUser[i].dataDiri.nik == dataPenduduk[cariIndex].nik &&
        dataUser[i].role == "admin") {
            adaAdmin = true;
            break;
        }
    }
    if (adaAdmin) {
        cout << "Tidak bisa hapus penduduk dengan akun admin!\n";
        return;
    }
    cout << "Data ditemukan: \n";
    Table table;
    table.add_row({"NIK", "Nama", "Alamat", "No Telp"});
    table.add_row({
    dataPenduduk[cariIndex].nik,
    dataPenduduk[cariIndex].nama,
    dataPenduduk[cariIndex].alamat,
    dataPenduduk[cariIndex].noTelp
    });
    table.column(0).format().width(6); // NIK
    table.column(1).format().width(16); // Nama
    table.column(2).format().width(16); // Alamat
    table.column(3).format().width(13); // No Telp
    cout << table << "\n";
    if (!konfirmasi("Yakin ingin hapus?")) {
        cout << "Batal!\n";
        return;
    }
    string nikHapus = dataPenduduk[cariIndex].nik;
    hapus(dataPenduduk, jumlahPenduduk, cariIndex);

    for (int i=0; i < jumlahUser; i++) {
        if (dataUser[i].dataDiri.nik == nikHapus) {
            if (i<indexUser) indexUser--;
            hapus(dataUser, jumlahUser, i);
            break;
        }
    }

    for (int i=0; i < jumlahSurat; i++) {
        if (dataSurat[i].nikPemohon == nikHapus) {
            hapus(dataSurat, jumlahSurat, i);
            i--;
```

Source Code 3.5.6 Prosedur Menghapus Penduduk

Proses penghapusan dimulai dengan mencari NIK, setelah ketemu data penduduk tersebut akan ditampilkan, sama dengan proses pengeditan, pengguna tidak dapat menghapus akun admin. Sebelum menghapus, pengguna diminta konfirmasi penghapusan.

Proses penghapusan dilakukan dengan memanggil prosedur 'hapus' dengan cara pergeseran. Sebelum itu NIK dari data penduduk yang dihapus akan disimpan di variabel 'nikHapus' kenapa? Dalam penghapusan penduduk, akun dan semua surat yang penduduk tersebut miliki juga akan dihapus.

Jika diperhatikan di kode penghapusan akun ada kondisi *if* jika indeks penduduk yang dihapus kurang dari indeks penduduk yang sedang *login*, maka nilai dari variabel 'indexUser' akan berkurang 1, ini untuk mencegah kesalahan indeks karena setelah pergeseran ada kemungkinan indeks penduduk tidak berubah. Penghapusan surat juga ada perintah tambahan yaitu 'i– -', sebelum 'i' pada perulangan *for* bertambah 'i' akan dikurangi terlebih dahulu, ini bertujuan untuk memastikan semua surat dari penduduk tersebut terhapus dan tidak ada yang terlewat.

```
void hapusAkun(User data[], int &jumlah, int &indexUser) {
    string inputUser;
    cout << "Masukkan username dari akun yang ingin dihapus: ";
    getline(cin, inputUser);
    int cariIndex = cari(data, jumlah, inputUser);
    if (cariIndex == -1) {
        cout << "Username tidak ditemukan!\n";
        return;
    }
    if (data[cariIndex].role == "admin") {
        cout << "Tidak bisa hapus akun admin!\n";
        return;
...
if (!konfirmasi("Yakin ingin hapus?")) {
        cout << "Batal hapus!\n";
        return;
    }
    hapus(data, jumlah, cariIndex);
    if (cariIndex < indexUser) {
        indexUser--;
```

Source Code 3.5.7 Prosedur Menghapus Akun

Proses ini sama persis dengan penghapusan data penduduk, cari akun berdasarkan *username* lalu panggil 'hapus' versi akun. Proses penghapusan sama seperti subbab sebelumnya.

```
void updateSuratAdmin(Surat data[], int jumlah) {
```

```
    string cariID;
    cout << "Masukkan ID surat dari surat yang ingin diubah:";
    getline(cin, cariID);
    int cariIndex = cari(data, jumlah, cariID);
    if (cariIndex == -1) {
        cout << "Surat tidak ditemukan!\n";
        return;
    }
    cout << "Data ditemukan: \n";
...
string opsiUbahSurat[] = {"Ubah status", "Ubah keterangan", "Kembali"};
    int pilihan = tampilMenu("Pilih perubahan\n", opsiUbahSurat, 3);
    switch(pilihan) {
        case 1: {
            int pilihStatus = tampilMenu("Pilih status\n", daftarStatus, 4);
// Tolong dicek lagi
            data[cariIndex].status = daftarStatus[pilihStatus-1];
            cout << "Status berhasil diganti!\n";
            break;
        }
        case 2: {
            cin.ignore();
            string isiKeterangan = inputOpsional("Isi keterangan surat
(kosongkan untuk '-'): ", 50);
            if (isiKeterangan == "") {
                data[cariIndex].keterangan = "-";
                cout << "Keterangan dikosongkan\n";
                break;
            }
            data[cariIndex].keterangan = isiKeterangan;
            cout << "Keterangan berhasil diubah!\n";
```

Source Code 3.5.8 Prosedur Mengedit Surat Versi Admin

Untuk melakukan pengeditan surat, cari surat berdasarkan ID, setelah ketemu pengguna dapat memilih perubahan yang diinginkan. Di perubahan status daftar akan ditampilkan menggunakan fungsi 'tampilMenu', pilihan disimpan di dalam 'daftarStatus[pilihStatus-1]' dan dilakukan dengan cara timpa. Perubahan keterangan juga dengan cara timpa. Khusus keterangan, *field* ini boleh dikosongkan dan mengisi nilainya dengan *strip*.

```
void hapusSuratAdmin(Surat data[], int &jumlah) {
    string cariID;
    cout << "Masukkan ID surat dari surat yang ingin diubah:";
    getline(cin, cariID);
    int cariIndex = cari(data, jumlah, cariID);
    if (cariIndex == -1) {
```

```
        cout << "Surat tidak ditemukan!\n";
        return;
    }
    cout << "Data ditemukan: \n";
...
if(!konfirmasi("Yakin ingin hapus?")) {
        cout << "Batal hapus!\n";
        return;
    }

    hapus(data, jumlah,cariIndex);
    cout << "Berhasil dihapus!\n";
```

Source Code 3.5.9 Prosedur Hapus Surat Admin

Pada prosedur ini, kita harus memasukkan ID surat yang ingin dihapus. Surat tersebut kemudian ditampilkan dan program meminta konfirmasi pengguna sebelum menghapus. Jika ya maka surat tersebut dihapus.

3.6 Daftar Fungsi Fitur Utama User

```
void gantiPassword(User data[], int indexUser) {
    string passwordTempo = inputStr("Masukkan password sekarang untuk
mengganti password");
    if (data[indexUser].pw == passwordTempo) {
        string passwordGanti = inputMin("Masukkan password baru: ", 3);
        string konfirmasi = inputStr("Konfirmasi password baru: ");
        if (passwordGanti == konfirmasi) {
            data[indexUser].pw = passwordGanti;
            cout << "Password berhasil diubah!\n";
            return;
        } else {
            cout << "Password salah!\n";
        }
    } else {
        cout << "Password salah!\n";
```

Source Code 3.6.1 Prosedur Mengganti Password

Prosedur ini memungkinkan pengguna dengan *role user* mengganti *password* mereka. Prosesnya masukkan *password* saat ini untuk verifikasi, setelah itu masukkan *password* baru dan masukkan *password* yang sama sekali lagi sebagai konfirmasi. Proses ini menggunakan banyak proses perbandingan.

```
void buatSurat(Surat dataSurat[], int &jumlahSurat, int &totalSurat, int
indexUser, User dataUser[]) {
    int pilihJenis = tampilMenu("Pilih Jenis\n", daftarJenis, 4);
```

```
    string jenisBaru = daftarJenis[pilihJenis-1];
    int pilihKeperluan = tampilMenu("Pilih keperluan\n", daftarKeperluan,
4);
    string keperluanBaru = daftarKeperluan[pilihKeperluan-1];
    string nikPemohonBaru = dataUser[indexUser].dataDiri.nik;
    string idBaru = "SRT" + to_string(++totalSurat);
    dataSurat[jumlahSurat] = {idBaru, nikPemohonBaru, jenisBaru,
keperluanBaru, "menunggu", "-"};
    jumlahSurat++;
```

Source Code 3.6.2 Prosedur Menambah Surat

Prosedur ini dikhususkan ke *user*, pengisian *field* 'jenisSurat' dan 'keperluan' menggunakan proses yang sama seperti update status surat di fungsi sebelumnya. Sementara 'nikPemohon' mengakses NIK dari 'indexUser' atau pengguna yang membuat surat tersebut. Pembuatan ID surat sama dengan pembuatan NIK, lalu simpan ke data surat dengan status 'menunggu' dan keterangan yang dikosongkan.

```
void editSuratUser(Surat dataSurat[], int jumlahSurat, int indexUser, User
dataUser[]) {
    string cariID;
    cout << "Masukkan ID surat dari surat yang ingin diubah: ";
    getline(cin, cariID);
    int cariIndex = cariSuratUser(dataSurat, jumlahSurat, cariID,
dataUser[indexUser].dataDiri.nik);
    if (cariIndex == -1) {
        cout << "Surat tidak ditemukan!\n";
        return;
    }
    if (!statusMenunggu(dataSurat, cariIndex)) {
        cout << "Surat tidak bisa diedit/dihapus!\n";
        return;
...
string opsiUbahSurat[] = {"Jenis surat", "Keperluan surat", "Kembali"};
    int pilihan = tampilMenu("Pilih perubahan\n", opsiUbahSurat, 3);
    switch(pilihan) {
        case 1: {
            int pilihJenis = tampilMenu("Pilih jenis\n", daftarJenis, 4);
            dataSurat[cariIndex].jenisSurat = daftarJenis[pilihJenis-1];
            cout << "Berhasil diubah!\n";
            break;
        }
        case 2: {
            int pilihKeperluan = tampilMenu("Pilih kepeluan",
daftarKeperluan, 4);
            dataSurat[cariIndex].keperluan =
daftarKeperluan[pilihKeperluan-1];
            cout << "Berhasil diubah!\n";
```

Source Code 3.6.3 Prosedur Pengeditan Surat Versi User

Cari terlebih dahulu surat yang ingin diubah melalui ID surat, jika surat tersebut memiliki status selain menunggu maka surat tidak dapat diedit. Perubahan hanya ada di jenis dan keperluan, proses pengeditan sama ketika menambah surat, pilih perubahan dalam daftar dan data lama akan ditimpa.

```
void hapusSuratUser(Surat dataSurat[], int &jumlahSurat, int indexUser, User
dataUser[]) {
    string cariID;
    cout << "Masukkan ID surat dari surat yang ingin diubah: ";
    getline(cin, cariID);
    int cariIndex = cariSuratUser(dataSurat, jumlahSurat, cariID,
dataUser[indexUser].dataDiri.nik);
    if (cariIndex == -1) {
        cout << "Surat tidak ditemukan!\n";
        return;
    }
    if (!statusMenunggu(dataSurat, cariIndex)) {
        cout << "Surat tidak bisa diedit/dihapus!\n";
        return;
...
if(!konfirmasi("Yakin ingin hapus?")) {
        cout << "Batal hapus!\n";
        return;
    }

    hapus(dataSurat, jumlahSurat, cariIndex);
    cout << "Berhasil dihapus!\n";
```

Source Code 3.6.4 Prosedur Hapus Surat Versi User

Cari surat melalui ID, jika status ‘menunggu’ maka penghapusan bisa dilakukan. Setelah konfirmasi baru surat dihapus dengan pergeseran.

3.7 Main

```
int main() {
    Penduduk dataPenduduk[100];
    User dataUser[100];
    Surat dataSurat[100];
    int jumlahPenduduk = 2;
    int jumlahUser = 2;
    int jumlahSurat = 2;
    int totalSurat = 2;
    int totalPenduduk = 2;
    int indexUser = -1;
```

```
    dataPenduduk[0] = {"PDK1", "radja", "gg bersatu", "08111"};
    dataPenduduk[1] = {"PDK2", "amat", "gg bersama", "08112"};

    dataUser[0] = {"r", "012", "admin", dataPenduduk[0]};
    dataUser[1] = {"a", "011", "user", dataPenduduk[1]};

    dataSurat[0] = {"SRT1", "PDK2", "Surat Keterangan Tidak Mampu",
"Keperluan Pendidikan", "diproses",
                    "Dalam pengecekkan"};
    dataSurat[1] = {"SRT2", "PDK2", "Surat Keterangan Tidak Mampu",
"Keperluan usaha", "menunggu", "-"};
```

Source Code 3.7.1 Inisialisasi Variabel dan Array

Sebelum masuk ke program utama, ada beberapa inisialisasi. Tiga *array* dari masing-masing kategori, penetapan jumlah dan total. Lalu variabel 'indexUser' menunjukkan indeks pengguna yang sedang *login*. Data *dummy* dari masing-masing kategori untuk mempermudah program.

```
while (true) {
        cout << "Selamat datang di Website Balai Kota!\n";
        string opsiAuth[] = {"Login", "Register", "Keluar"};
        int pilihan = tampilMenu("Silahkan login untuk lanjut\n", opsiAuth,
3);
        switch(pilihan) {
```

Source Code 3.7.2 Menu Autentikasi

Di dalam *loop while* utama pengguna harus *login* untuk lanjut atau registrasi.

```
if (dataUser[indexUser].role == "admin") {
                    while (true) {
                        string opsiAdmin[] = {"Data penduduk", "Data akun",
"Data surat", "Log-out"};
                        int pilihanAdmin = tampilMenu("Menu Admin",
opsiAdmin, 4);
                        switch(pilihanAdmin) {
```

Source Code 3.7.3 Menu Admin

Jika saat *login* kesempatan pengguna habis maka program langsung berhenti dengan 'return 0'. Jika pengguna adalah admin ia bisa memilih menu yang diinginkan, penduduk, akun, atau surat.

```
case 1: {
                                while (true) {
                                    string opsiPenduduk[] = {"Lihat",
"Tambah", "Edit", "Hapus", "Kembali"};
                                    int pilihanPenduduk = tampilMenu("Menu
Penduduk", opsiPenduduk, 5);
                                    cin.ignore();
```

```
                                    switch(pilihanPenduduk) {
                                        case 1: tampilPenduduk(dataPenduduk,
jumlahPenduduk); break;
                                        case 2: tambahPenduduk(dataPenduduk,
jumlahPenduduk, totalPenduduk); break;
                                        case 3: editPenduduk(dataPenduduk,
jumlahPenduduk, dataUser, jumlahUser); break;
                                        case 4:
hapusPendudukAdmin(dataPenduduk, jumlahPenduduk, dataUser, jumlahUser,
dataSurat, jumlahSurat, indexUser); break;
...
case 2: {
                                while (true) {
                                    string opsiAkun [] = {"Lihat", "Hapus",
"Kembali"};
                                    int pilihanAkun = tampilMenu("Menu
Akun", opsiAkun, 3);
                                    cin.ignore();
                                    switch(pilihanAkun) {
                                        case 1: tampilAkun(dataUser,
jumlahUser); break;
                                        case 2: hapusAkun(dataUser,
jumlahUser, indexUser); break;
...
case 3: {
                                while (true) {
                                    string opsiSurat[] = {"Lihat", "Update",
"Hapus", "Kembali"};
                                    int pilihanSurat = tampilMenu("Menu
Surat", opsiSurat, 4);
                                    cin.ignore();
                                    switch(pilihanSurat) {
                                        case 1: tampilSurat(dataSurat,
jumlahSurat); break;
                                        case 2: updateSuratAdmin(dataSurat,
jumlahSurat); break;
                                        case 3: hapusSuratAdmin(dataSurat,
jumlahSurat); break;
...
case 4: {
                                cout << "Log-out berhasil!\n";
                                indexUser = -1;
                                break;
                            }
                        }
                        if (pilihanAdmin == 4) break;
```

Source Code 3.7.4 Proses di Setiap Menu

Di menu penduduk admin dapat melihat daftar, mengedit, dan menghapus data penduduk. Di menu akun dapat melihat daftar akun dan menghapus data akun. Di menu surat admin dapat melihat daftar surat, update dan menghapus surat. Memilih pilihan keempat aitu *log-out* akan mengembalikan pengguna ke menu *auth* dan 'indexUser' kembali ke -1. Semua proses CRUD akan memanggil fungsi yang sesuai yang sudah dijelaskan di subbab sebelumnya.

```
else {
                    while (true) {
                        string opsiUser[] = {"Data Sendiri", "Ganti
Password", "Surat Menyurat", "Log-out"};
                        int pilihanUser = tampilMenu("Menu User", opsiUser,
4);
                        switch(pilihanUser) {
```

Source Code 3.7.5 Menu User

Role user hanya bisa melihat data sendiri, mengganti *password*, dan CRUD surat.

```
case 1: {
                                cout << "NIK: " <<
dataUser[indexUser].dataDiri.nik << "\n";
                                cout << "Nama: " <<
dataUser[indexUser].dataDiri.nama << "\n";
                                cout << "Alamat: " <<
dataUser[indexUser].dataDiri.alamat << "\n";
                                cout << "No Telp: " <<
dataUser[indexUser].dataDiri.noTelp << "\n";
```

Source Code 3.7.6 Menampilkan Data Sendiri

Menampilkan data sendiri menggunakan cara *cout* sederhana. *Field* yang ditampilkan sebenarnya berasal dari *struct* 'Penduduk' namun diakses melalui *nested struct* dalam *struct* 'User'.

```
 case 2: gantiPassword(dataUser, indexUser); break;
                            case 3: {
                                while (true) {
                                    string opsiSuratUser[] = {"Lihat Surat",
"Buat Surat", "Edit Surat", "Hapus Surat", "Kembali"};
                                    int pilihanSuratUser = tampilMenu("Menu
surat", opsiSuratUser, 5);
                                    cin.ignore();
                                    switch(pilihanSuratUser) {
                                        case 1: tampilSuratByNIK(dataSurat,
jumlahSurat, dataUser[indexUser].dataDiri.nik); break;
                                        case 2: buatSurat(dataSurat,
```

```
jumlahSurat, totalSurat, indexUser, dataUser); break;
                                        case 3: editSuratUser(dataSurat,
jumlahSurat, indexUser, dataUser); break;
                                        case 4: hapusSuratUser(dataSurat,
jumlahSurat, indexUser, dataUser); break;
```

Source Code 3.7.7 Menu User Lainnya

Menu yang lain sudah memiliki fungsinya sendiri, jadi cukup panggil prosedur dan fungsi yang sesuai.

```
case 2: {
                cin.ignore();
                registarsi(dataUser, jumlahUser, dataPenduduk,
jumlahPenduduk); break;
            }
            case 3: {
                return 0;
```

Source Code 3.7.8 Pilihan Regis dan Keluar Program

Panggil fungsi ‘registrasi’ dengan argumen data *user* dan penduduk serta jumlah keduanya. Pilihan 3 akan mengakhiri program.

4. Hasil Output

```
Selamat datang di Website Balai Kota!

Silahkan login untuk melanjutkan
1. Login
2. Register
3. Keluar
Pilih:
```

Gambar 4.1 Tampilan Auth

```
Pilih: 2
Masukkan username baru: am
Masukkan password baru: 013
Masukkan nama lengkap: amat
Ditemukan 2 penduduk dengan nama sama:
1. amat - PDK3
2. amat - PDK4
Pilih nomor: 1
Registrasi berhasil!
Nik kamu: PDK3
```

Gambar 4.2 Registrasi dengan Nama Sama

```
Login berhasil! Selamat datang, r!


Pilih menu field:
1. Data Penduduk
2. Data akun
3. Data surat
4. Log-out
```

Gambar 4.3 Menu Admin

```
Pilih menu penduduk:
1. Lihat daftar penduduk
2. Tambah penduduk
3. Edit data penduduk
4. Hapus penduduk
5. Kembali
```

Gambar 4.4 Menu Penduduk

```
Pilih: 1
1. NIK: PDK1, Nama: radja, Alamat: gg bersatu, Nomor Telepon: 08111
2. NIK: PDK2, Nama: amat, Alamat: gg bersama, Nomor Telepon: 08112
3. NIK: PDK3, Nama: amat, Alamat: gg batu, Nomor Telepon: 08113
4. NIK: PDK4, Nama: amat, Alamat: gg putih, Nomor Telepon: 08114
```

Gambar 4.5 Daftar Penduduk

```
Pilih: 2
Masukkan nama: Omar
Masukkan alamat: gg bersatu
Masukkan nomor telepon: 08115
Berhasil ditambahkan!
```

Gambar 4.6 Tambah Penduduk

```
Pilih: 3
Masukkan NIK dari data penduduk yang ingin diubah: PDK5
NIK: PDK5, Nama: Omar, Alamat: gg bersatu, Nomor Telepon: 08115

Pilih perubahan:
1. Ubah nama penduduk
2. Ubah alamat penduduk
3. Ubah no telepon penduduk
4. Kembali
Pilih: 2
Alamat baru: gg matahari
Berhasil diubah!
```

Gambar 4.7 Mengedit Data Penduduk

```
Pilih: 4
Masukkan NIK dari data penduduk yang ingin dihapus: PDK4
NIK: PDK4, Nama: amat, Alamat: gg putih, Nomor Telepon: 08114
Konfirmasi hapus? (y/n): y
Berhasil dihapus!
```

Gambar 4.8 Menghapus Penduduk

```
Pilih: 1
1. NIK: PDK1, Nama: radja, Alamat: gg bersatu, Nomor Telepon: 08111
2. NIK: PDK2, Nama: amat, Alamat: gg bersama, Nomor Telepon: 08112
3. NIK: PDK3, Nama: amat, Alamat: gg batu, Nomor Telepon: 08113
4. NIK: PDK5, Nama: Omar, Alamat: gg matahari, Nomor Telepon: 08115
```

Gambar 4.9 Daftar Penduduk Setelah Edit dan Hapus

```
Pilih menu akun:
1. Lihat daftar akun
2. Hapus akun
3. Kembali
```

Gambar 4.10 Tampilan Menu Akun

```
Pilih: 1
1. Username: r, Role: admin, NIK: PDK1, Nama: radja
2. Username: a, Role: user, NIK: PDK2, Nama: amat
3. Username: am, Role: user, NIK: PDK3, Nama: amat
```

Gambar 4.11 Daftar Akun

```
Pilih menu surat:
1. Lihat daftar surat
2. Update surat
3. Hapus surat
4. Kembali
Pilih: 1
1. ID surat: SRT1, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan:
Keperluan Pendidikan, Status: diproses, Keterangan: Dalam pengecekkan
2. ID surat: SRT2, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan:
Keperluan usaha, Status: menunggu, Keterangan: -
```

Gambar 4.12 Menu Surat dan Lihat Daftar Surat

```
Pilih: 2
Masukkan ID dari surat yang ingin diupdate: SRT1
ID surat: SRT1, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan: Kep
erluan Pendidikan, Status: selesai, Keterangan: Dalam pengecekkan

Pilih perubahan:
1. Ubah status
2. Ubah keterangan
3. Kembali
Pilih: 2
Isi keterangan surat: Sudah dikirimkan
Keterangan berhasil diubah!
Pilih menu surat:
1. Lihat daftar surat
2. Update surat
3. Hapus surat
4. Kembali
Pilih: 3
Masukkan ID dari surat yang ingin dihapus: SRT2
ID surat: SRT2, Jenis: Surat Keterangan Tidak Mampu, Status: menunggu
Konfirmasi hapus? (y/n): y
Berhasil dihapus!
```

Gambar 4.13 Edit dan Hapus Surat

```
Silahkan login untuk melanjutkan
Username: am
Password: 013

Login berhasil! Selamat datang, am!


Pilih menu:
1. Data sendiri
2. Ganti password
3. Surat menyurat
4. Log-out
```

Gambar 4.14 Menu User

```
Data diri kamu
NIK: PDK3
Nama: amat
Alamat: gg batu
Nomor Telepon: 08113
```

Gambar 4.15 Data Sendiri

```
Pilih: 2
Masukkan password sekarang untuk mengganti password:
013
Masukkan password baru: 015
Konfirmasi password baru: 015
Password berhasil diubah!
```

Gambar 4.16 Mengubah Password

```
Pilih: 2
Buat surat: Pilih jenis:
1. Surat Keterangan Domisili
2. Surat Keterangan Tidak Mampu
3. Surat Pengantar KTP
4. Surat Keterangan Kelahiran
Pilih: 3
Pilih keperluan:
1. Melamar pekerjaan
2. Keperluan pendidikan
3. Keperluan pernikahan
4. Lainnya
Pilih: 4
Surat berhasil ditambahkan!
```

Gambar 4.17 Buat Surat

```
Masukkan ID surat yang ingin diubah:
SRT4
Pilih perubahan:
1. Jenis surat
2. Keperluan surat
3. Kembali
Pilih: 2
Pilih keperluan:
1. Melamar pekerjaan
2. Keperluan pendidikan
3. Keperluan pernikahan
4. Lainnya
Pilih: 2
Keperluan surat berhasil diubah!
```

Gambar 4.18 Edit Surat User

```
Daftar surat:
2. ID: SRT4, Jenis: Surat Pengantar KTP, Keperluan: Keperluan pendidikan, Status: menunggu,
Keterangan: -
```

Gambar 4.19 Daftar Surat User

```
Selamat datang di Website Balai Kota!
+-----------------------------+
| Silahkan login untuk lanjut |
+-----------------------------+
| 1. Login                    |
+-----------------------------+
| 2. Register                 |
+-----------------------------+
| 3. Keluar                   |
+-----------------------------+
Pilih:
```

Gambar 4.20 Tampilan dengan Tabel di Menu Awal

```
+----+------+------+----------------+-----------+--------+-------------------+
| No | ID   | NIK  | Jenis Surat    | Keperluan | Status | Keterangan        |
|    | Sur- | Pem- |                |           |        |                   |
|    | at   | ohon |                |           |        |                   |
+----+------+------+----------------+-----------+--------+-------------------+
| 1  | SRT1 | PDK2 | Surat          | Keperluan | dipro- | Dalam pengecekkan |
|    |      |      | Keterangan     | Pendidik- | ses    |                   |
|    |      |      | Tidak Mampu    | an        |        |                   |
+----+------+------+----------------+-----------+--------+-------------------+
| 2  | SRT2 | PDK2 | Surat          | Keperluan | menun- | -                 |
|    |      |      | Keterangan     | usaha     | ggu    |                   |
|    |      |      | Tidak Mampu    |           |        |                   |
+----+------+------+----------------+-----------+--------+-------------------+
```

Gambar 4.21 Tampilan Daftar Surat dengan Tabel

5. Langkah-langkah GIT

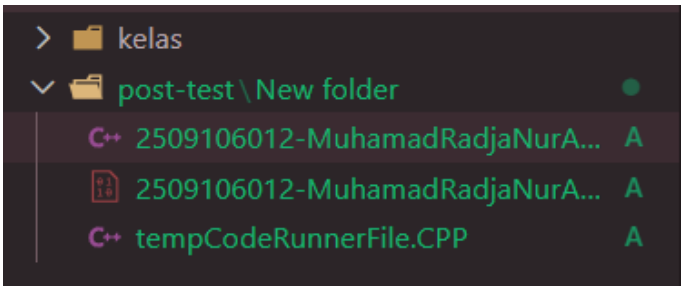
Sebelum melakukan proses *Git*, kita harus memiliki akun GitHub dan membuat *repository* baru. Lalu siapkan *folder* yang akan di git dan buka Command Prompt, PowerShell, atau semacamnya untuk melakukan *git*. Karena sudah melakukan *git init* dan *git remote*, maka prosesnya tidak perlu ditampilkan lagi.

5.1 Git Add

```
PS D:\praktikum-apl> git add .
```

Gambar 5.1.1 Proses Git Add

Git add berfungsi untuk memindah semua perubahan di area kerja ke indeks, ada dua cara untuk melakukan *git add*, yang pertama dengan mengetikkan nama *file* yang ingin di add. Jika kalian ingin menambahkan banyak *file* sekaligus, ketik "." Setelah *add* maka semua *file* yang di dalam area kerja akan ditambahkan.



Gambar 5.1.2 Setelah Git Add

5.2 Git Commit

```
PS D:\praktikum-apl> git commit -m "finish pt 1"
[master (root-commit) e8f9088] finish pt 1
 3 files changed, 98 insertions(+)
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.CPP
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.exe
 create mode 100644 post-test/New folder/tempCodeRunnerFile.CPP
```

Gambar 5.2.1 Proses Commit

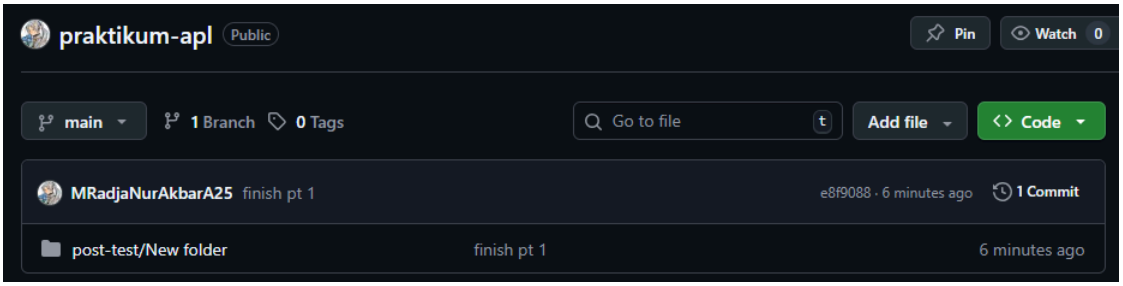
Git commit berfungsi untuk mengkonfirmasi setiap perubahan pada *repository* kalian dengan mengetikkan "git commit -m 'pesan yang ingin ditulis' " .

5.3 Git Push

```
PS D:\praktikum-apl> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 667.93 KiB | 7.95 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Gambar 5.3.1 Git Push

Git push berfungsi untuk mengunggah *file* lokal tadi ke GitHub kalian dengan mengetikkan "git push -u origin main. Kembali ke *repository* di GitHub kalian, klik *refresh* dan *file* sudah terunggah.



Gambar 5.3.2 File Terunggah